Introduction to Machine Learning

A Comprehensive Guide for Artificial Intelligence Students

Third Year, First Semester
Bachelor in Artificial Intelligence
Academic Year 2023-2024

Department of Artificial Intelligence
October 7, 2025

Preface

Welcome to the fascinating world of machine learning! This textbook is designed specifically for third-year students in the Bachelor of Artificial Intelligence program. Machine learning represents one of the most transformative technologies of our time, enabling computers to learn from data and make intelligent decisions without being explicitly programmed for every scenario.

How to Use This Book

Each chapter in this textbook corresponds to one or two weeks of course material. The structure of each chapter includes:

- **Learning Objectives:** Clear goals for what you should understand
- **Core Concepts:** Theoretical foundations with mathematical formulations
- **Practical Implementation:** Python code examples using popular libraries
- **Real-World Applications:** Case studies and industry examples
- **Exercises:** Problems to reinforce your understanding
- **Lab Work:** Hands-on projects to build practical skills

Prerequisites

Students should have completed:

- Programming Fundamentals (Python preferred)
- Linear Algebra
- Probability and Statistics
- Calculus I & II

Contents

Preface **3**

I Foundations **9**

1 Introduction to Machine Learning **11**
1.1 Learning Objectives . . . 11
1.2 What is Machine Learning? . . . 11
1.2.1 Definition and Core Concept . . . 11
1.2.2 Formal Definition . . . 11
1.2.3 Why Machine Learning? . . . 12
1.3 Types of Machine Learning . . . 12
1.3.1 Supervised Learning . . . 12
1.3.2 Unsupervised Learning . . . 13
1.3.3 Reinforcement Learning . . . 14
1.4 The Machine Learning Pipeline . . . 15
1.4.1 Complete ML Workflow . . . 15
1.4.2 Detailed Pipeline Steps . . . 15
1.5 Setting Up Your ML Environment . . . 17
1.5.1 Python Environment Setup . . . 17
1.5.2 Essential Libraries Overview . . . 17
1.6 Your First Complete ML Project . . . 18
1.6.1 Project: Iris Species Classification . . . 18

2 Mathematical Foundations for Machine Learning **21**
2.1 Learning Objectives . . . 21
2.2 Linear Algebra Essentials . . . 21
2.2.1 Vectors and Operations . . . 21
2.2.2 Matrices and Operations . . . 22
2.2.3 Special Matrices . . . 23
2.3 Probability and Statistics . . . 23
2.3.1 Probability Basics . . . 23
2.3.2 Statistical Measures . . . 25
2.4 Calculus for Machine Learning . . . 26
2.4.1 Derivatives and Gradients . . . 26
2.4.2 Chain Rule . . . 27
2.4.3 Optimization with Gradient Descent . . . 27
2.4.4 Multivariable Optimization . . . 27
2.5 Information Theory for ML . . . 29
2.5.1 Entropy . . . 29

2.5.2 Cross-Entropy . . . 29
2.5.3 KL Divergence . . . 29
2.6 Chapter Summary . . . 31
2.7 Exercises . . . 31
2.7.1 Mathematical Exercises . . . 31
2.7.2 Programming Exercises . . . 31

3 Data Preprocessing and Feature Engineering 33
3.1 Learning Objectives . . . 33
3.2 Data Quality and Cleaning . . . 33
3.2.1 Understanding Data Quality Issues . . . 33
3.2.2 Data Profiling and Exploration . . . 35
3.2.3 Handling Missing Data . . . 37
3.2.4 Outlier Detection and Treatment . . . 40
3.3 Feature Scaling and Transformation . . . 43
3.3.1 Feature Scaling Techniques . . . 43
3.3.2 Feature Transformation Techniques . . . 45
3.4 Feature Engineering . . . 48
3.4.1 Creating New Features . . . 48
3.5 Feature Selection . . . 52
3.5.1 Feature Selection Methods . . . 52
3.6 Chapter Summary . . . 57
3.7 Exercises . . . 58
3.7.1 Conceptual Questions . . . 58
3.7.2 Programming Exercises . . . 58
3.7.3 Lab Assignment . . . 59

4 Classification Algorithms 61
4.1 Learning Objectives . . . 61
4.2 Introduction to Classification . . . 61
4.2.1 Types of Classification Problems . . . 61
4.2.2 Key Concepts . . . 61
4.3 Logistic Regression . . . 62
4.3.1 Theory and Mathematical Foundation . . . 62
4.3.2 Binary Logistic Regression Implementation . . . 62
4.3.3 Multi-class Logistic Regression . . . 66
4.4 Decision Trees . . . 68
4.4.1 Theory and Concepts . . . 68
4.4.2 Decision Tree Implementation . . . 68
4.4.3 Pruning and Regularization . . . 74
4.5 Support Vector Machines (SVM) . . . 77
4.5.1 Theory and Mathematical Foundation . . . 78
4.5.2 SVM Implementation . . . 78
4.5.3 Kernel Methods and Hyperparameter Tuning . . . 81
4.6 Ensemble Methods . . . 85
4.6.1 Introduction to Ensemble Learning . . . 86
4.6.2 Random Forests . . . 86
4.6.3 Boosting Methods . . . 91
4.7 Handling Imbalanced Classification . . . 96
4.7.1 Imbalanced Data Challenges . . . 96
4.7.2 Techniques for Imbalanced Data . . . 96

4.8 Model Evaluation and Comparison 101
4.8.1 Classification Metrics 102
4.8.2 Algorithm Comparison Framework 102
4.9 Chapter Summary 108
4.10 Exercises 109
4.10.1 Conceptual Questions 109
4.10.2 Programming Exercises 109
4.10.3 Lab Assignment: Comprehensive Classification Project 110

Part I

Foundations

Chapter 1

Introduction to Machine Learning

1.1 Learning Objectives

Learning Objectives

By the end of this chapter, you will be able to:

- Define machine learning and distinguish it from traditional programming
- Identify different types of machine learning problems
- Understand the machine learning pipeline
- Set up a Python environment for machine learning
- Implement your first machine learning model

1.2 What is Machine Learning?

1.2.1 Definition and Core Concept

Definition

Machine Learning (ML) is a subset of artificial intelligence that enables systems to learn and improve from experience without being explicitly programmed. Instead of following pre-written rules, ML algorithms build mathematical models based on training data to make decisions or predictions.

Traditional Programming vs Machine Learning:

- **Traditional Programming:** Input + Rules → Output
- **Machine Learning:** Input + Output → Rules

1.2.2 Formal Definition

Tom Mitchell (1997) provides a formal definition: "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."

Example

Email Spam Filter:

- **Task (T):** Classifying emails as spam or not spam
- **Experience (E):** A dataset of emails labeled as spam or not spam
- **Performance (P):** Percentage of emails correctly classified

1.2.3 Why Machine Learning?

Machine learning excels in scenarios where:

1. Complex patterns exist: Problems too complex for traditional rule-based systems
2. Large amounts of data: Patterns that humans cannot easily detect
3. Adaptability required: Systems that need to adapt to new patterns
4. Scale needed: Solutions that must work across millions of instances

1.3 Types of Machine Learning

1.3.1 Supervised Learning

In supervised learning, the algorithm learns from labeled training data. Each training example consists of an input and the desired output.

Mathematical Formulation:

Given a training set $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$ where:

- $x_i \in X$ is the input (features)
- $y_i \in Y$ is the output (label)

The goal is to learn a function $f : X \to Y$ that can predict the output for new, unseen inputs.

Types of Supervised Learning:

1. **Classification:** Predicting discrete labels
   - Binary Classification: $Y = \{0, 1\}$ or $Y = \{-1, +1\}$
   - Multi-class Classification: $Y = \{1, 2, \ldots, k\}$
2. **Regression:** Predicting continuous values
   - $Y = \mathbb{R}$ (real numbers)

Example

[First Classifier]

```python
# Import necessary libraries
import numpy as np
import pandas as pd
```

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load the iris dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create and train the model
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)

# Make predictions
y_pred = clf.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

Listing 1.1: First Machine Learning Classifier

1.3.2 Unsupervised Learning

Unsupervised learning works with unlabeled data to discover hidden patterns or structures.

Mathematical Formulation:

Given a dataset $\{x_1, x_2, \ldots, x_n\}$ where $x_i \in X$, the goal is to discover structure in the data without labels.

Common Tasks:

1. **Clustering:** Group similar data points
   - Objective: Partition data into k clusters
   - Example: Customer segmentation
2. **Dimensionality Reduction:** Reduce the number of features
   - Objective: Find lower-dimensional representation
   - Example: Data visualization
3. **Anomaly Detection:** Identify unusual data points
   - Objective: Detect outliers
   - Example: Fraud detection

Example

[Clustering Example]

```python
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Generate sample data
from sklearn.datasets import make_blobs
X, _ = make_blobs(n_samples=300, centers=4, n_features=2, random_state=42)

# Apply K-means clustering
kmeans = KMeans(n_clusters=4, random_state=42)
clusters = kmeans.fit_predict(X)

# Visualize the results
plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            marker='x', s=200, linewidths=3, color='red')
plt.title('K-Means Clustering Example')
plt.show()
```

Listing 1.2: K-Means Clustering Example

1.3.3 Reinforcement Learning

Reinforcement learning involves an agent learning to make decisions by interacting with an environment.

Key Components:

- **Agent:** The learner or decision maker
- **Environment:** The world the agent interacts with
- **State (s):** Current situation of the agent
- **Action (a):** Choices available to the agent
- **Reward (r):** Feedback from the environment
- **Policy (π):** Strategy for choosing actions

Mathematical Formulation:

The goal is to learn a policy $\pi : S \to A$ that maximizes the expected cumulative reward:

$$J(\pi) = \mathbb{E}\left[\sum_{t=0}^{T} \gamma^t r_t\right]$$

where $\gamma \in [0, 1]$ is the discount factor.

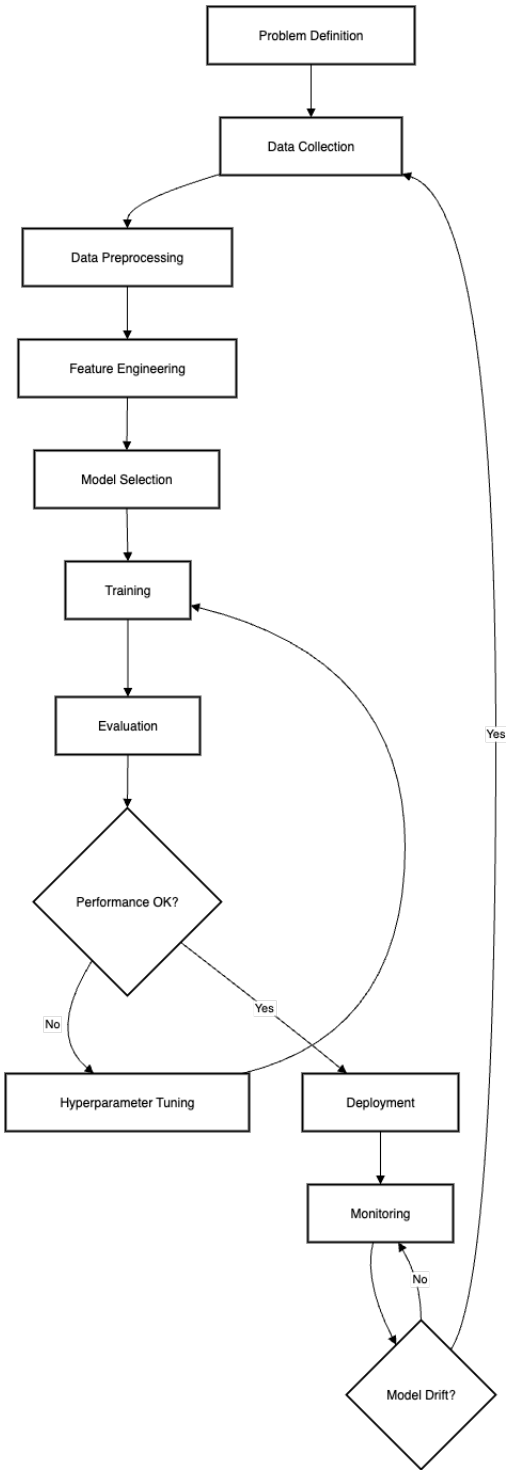

Figure 1.1: Machine Learning Pipeline Workflow

1.4 The Machine Learning Pipeline

1.4.1 Complete ML Workflow

1.4.2 Detailed Pipeline Steps

1. **Problem Definition**

- Identify the business problem
- Determine if ML is appropriate
- Define success metrics

2. **Data Collection**
   - Gather relevant data
   - Ensure data quality
   - Consider data privacy and ethics
3. **Data Preprocessing**
   - Handle missing values
   - Remove duplicates
   - Deal with outliers
4. **Feature Engineering**
   - Create new features
   - Transform existing features
   - Select relevant features
5. **Model Selection**
   - Choose appropriate algorithms
   - Consider computational constraints
   - Balance complexity and performance
6. **Training**
   - Split data (train/validation/test)
   - Fit model to training data
   - Monitor training progress
7. **Evaluation**
   - Use appropriate metrics
   - Check for overfitting/underfitting
   - Validate on unseen data
8. **Deployment**
   - Integrate into production system
   - Set up monitoring
   - Plan for updates

1.5 Setting Up Your ML Environment

1.5.1 Python Environment Setup

```bash
#!/bin/bash

# Create a virtual environment
python -m venv ml_env

# Activate the environment
# On Windows: ml_env\Scripts\activate
# On macOS/Linux: source ml_env/bin/activate

# Install essential packages
pip install numpy pandas matplotlib seaborn scikit-learn jupyter
```

Listing 1.3: Setting up Python Environment

1.5.2 Essential Libraries Overview

```python
# NumPy - Numerical computing
import numpy as np

# Create arrays
arr = np.array([1, 2, 3, 4, 5])
matrix = np.array([[1, 2], [3, 4]])

# Basic operations
mean = np.mean(arr)
std = np.std(arr)
dot_product = np.dot(matrix, matrix.T)

# Pandas - Data manipulation
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'feature_1': [1, 2, 3, 4],
    'feature_2': [5, 6, 7, 8],
    'label': ['A', 'B', 'A', 'B']
})

# Basic operations
df.describe()  # Statistical summary
df.groupby('label').mean()  # Group operations

# Matplotlib - Visualization
import matplotlib.pyplot as plt

# Create plots
plt.figure(figsize=(10, 6))
plt.subplot(1, 2, 1)
plt.hist(np.random.normal(0, 1, 1000), bins=30)
plt.title('Normal Distribution')
plt.subplot(1, 2, 2)
plt.scatter(np.random.randn(100), np.random.randn(100))
plt.title('Scatter Plot')
plt.show()

```

```python
# Scikit-learn - Machine Learning
from sklearn import datasets, model_selection, preprocessing

# Load data
iris = datasets.load_iris()

# Preprocessing
scaler = preprocessing.StandardScaler()
X_scaled = scaler.fit_transform(iris.data)

# Split data
X_train, X_test, y_train, y_test = model_selection.train_test_split(
    X_scaled, iris.target, test_size=0.2
)
```

Listing 1.4: Essential Python Libraries for ML

1.6 Your First Complete ML Project

1.6.1 Project: Iris Species Classification

Let's build a complete machine learning project from start to finish.

```python
# Step 1: Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# Step 2: Load and explore data
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df['species_name'] = df['species'].map({0: 'setosa', 1: 'versicolor', 2: '
    virginica'})

print("Dataset shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
print("\nDataset info:")
print(df.info())
print("\nStatistical summary:")
print(df.describe())

# Step 3: Data visualization
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Distribution of features
for idx, col in enumerate(iris.feature_names):
    ax = axes[idx // 2, idx % 2]
    for species in df['species'].unique():
        subset = df[df['species'] == species]
```

```
        ax.hist(subset[col], alpha=0.5, label=iris.target_names[species])
    ax.set_xlabel(col)
    ax.set_ylabel('Frequency')
    ax.legend()
    ax.set_title(f'Distribution of {col}')

plt.tight_layout()
plt.show()

# Step 4: Data preprocessing
X = iris.data
y = iris.target

# Scale the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# Step 5: Model training and evaluation
models = {
    'Logistic Regression': LogisticRegression(random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(random_state=42)
}

results = {}

for name, model in models.items():
    # Train the model
    model.fit(X_train, y_train)

    # Make predictions
    y_pred = model.predict(X_test)

    # Calculate accuracy
    accuracy = accuracy_score(y_test, y_pred)
    results[name] = accuracy

    print(f"\n{name} Results:")
    print(f"Accuracy: {accuracy:.3f}")
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Step 6: Compare models
print("\nModel Comparison:")
for name, accuracy in results.items():
    print(f"{name}: {accuracy:.3f}")

# Step 7: Cross-validation for best model
best_model_name = max(results, key=results.get)
best_model = models[best_model_name]

cv_scores = cross_val_score(best_model, X_scaled, y, cv=5)
print(f"\nCross-validation scores for {best_model_name}: {cv_scores}")
print(f"Mean CV accuracy: {cv_scores.mean():.3f} (+/- {cv_scores.std() * 2:.3f})
    ")
```

Listing 1.5: Complete Iris Classification Project

Chapter 2

Mathematical Foundations for Machine Learning

2.1 Learning Objectives

Learning Objectives

By the end of this chapter, you will:

- Master essential linear algebra concepts for ML
- Understand probability and statistics in ML context
- Apply calculus concepts to optimization problems
- Implement mathematical concepts in Python
- Understand information theory measures used in ML

2.2 Linear Algebra Essentials

2.2.1 Vectors and Operations

Definition

A **vector** is an ordered array of numbers. In ML, vectors represent data points or features.

Mathematical Notation:

A vector $\mathbf{x} \in \mathbb{R}^n$ is written as:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Vector Operations:

1. **Addition:** $\mathbf{x} + \mathbf{y} = [x_1 + y_1, x_2 + y_2, ..., x_n + y_n]^T$

2. **Scalar Multiplication:** $c\mathbf{x} = [cx_1, cx_2, ..., cx_n]^T$
3. **Dot Product:** $\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^{n} x_i y_i$
4. **Norm:** $\|\mathbf{x}\|_p = (\sum_{i=1}^{n} |x_i|^p)^{1/p}$

Example

[Vector Operations in Python]

```python
import numpy as np

# Creating vectors
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])

# Vector operations
addition = x + y
print(f"Addition: {addition}")

scalar_mult = 2 * x
print(f"Scalar multiplication: {scalar_mult}")

dot_product = np.dot(x, y)
print(f"Dot product: {dot_product}")

# Different norms
l1_norm = np.linalg.norm(x, ord=1)
l2_norm = np.linalg.norm(x, ord=2)
inf_norm = np.linalg.norm(x, ord=np.inf)

print(f"L1 norm: {l1_norm}")
print(f"L2 norm: {l2_norm}")
print(f"Infinity norm: {inf_norm}")
```

Listing 2.1: Vector Operations

2.2.2 Matrices and Operations

Definition

A **matrix** is a 2D array of numbers. In ML, matrices represent datasets or transformations.

Matrix Operations:

1. **Matrix Multiplication:** For $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$:

$$(\mathbf{AB})_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$$

2. **Transpose:** $(\mathbf{A}^T)_{ij} = a_{ji}$
3. **Inverse:** $\mathbf{AA}^{-1} = \mathbf{I}$
4. **Eigenvalues and Eigenvectors:** $\mathbf{Ax} = \lambda \mathbf{x}$

Example

[Matrix Operations in Python]

```python
# Creating matrices
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Matrix operations
matrix_mult = np.dot(A, B)
print(f"Matrix multiplication:\n{matrix_mult}")

transpose = A.T
print(f"Transpose:\n{transpose}")

# Inverse (if exists)
if np.linalg.det(A) != 0:
    inverse = np.linalg.inv(A)
    print(f"Inverse:\n{inverse}")
    print(f"Verification A * A^(-1):\n{np.dot(A, inverse)}")

# Eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(A)
print(f"Eigenvalues: {eigenvalues}")
print(f"Eigenvectors:\n{eigenvectors}")
```

Listing 2.2: Matrix Operations

2.2.3 Special Matrices

- **Identity Matrix:**

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}$$

- **Diagonal Matrix:**

$$\text{diag}(d_1, d_2, ..., d_n) = \begin{bmatrix} d_1 & 0 & \cdots & 0 \\ 0 & d_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_n \end{bmatrix}$$

- **Symmetric Matrix:** $\mathbf{A} = \mathbf{A}^T$
- **Positive Definite Matrix:** $\mathbf{x}^T\mathbf{A}\mathbf{x} > 0$ for all $\mathbf{x} \neq 0$

2.3 Probability and Statistics

2.3.1 Probability Basics

Probability Rules:

1. **Sum Rule:** $P(X) = \sum_Y P(X, Y)$
2. **Product Rule:** $P(X, Y) = P(X|Y)P(Y)$

3. **Bayes' Theorem:** $P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$

Common Distributions:

1. **Gaussian (Normal) Distribution:**

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

2. **Bernoulli Distribution:**

$$P(X = x) = p^x(1-p)^{1-x}$$

3. **Multinomial Distribution:**

$$P(x_1, ..., x_k) = \frac{n!}{x_1!...x_k!} p_1^{x_1} ... p_k^{x_k}$$

Example

[Probability Distributions Visualization]

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Gaussian distribution
mu, sigma = 0, 1
x = np.linspace(-4, 4, 100)
gaussian_pdf = stats.norm.pdf(x, mu, sigma)

# Visualize distributions
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Gaussian
axes[0, 0].plot(x, gaussian_pdf)
axes[0, 0].fill_between(x, gaussian_pdf, alpha=0.3)
axes[0, 0].set_title('Gaussian Distribution ($\mu=0$, $\sigma=1$)')
axes[0, 0].set_xlabel('x')
axes[0, 0].set_ylabel('Probability Density')

# Bernoulli
p = 0.3
bernoulli_x = [0, 1]
bernoulli_pmf = [1-p, p]
axes[0, 1].bar(bernoulli_x, bernoulli_pmf, width=0.3)
axes[0, 1].set_title(f'Bernoulli Distribution (p={p})')
axes[0, 1].set_xlabel('x')
axes[0, 1].set_ylabel('Probability')

# Multiple Gaussians with different parameters
axes[1, 0].plot(x, stats.norm.pdf(x, 0, 1), label='$\mu=0$, $\sigma=1$')
axes[1, 0].plot(x, stats.norm.pdf(x, 0, 0.5), label='$\mu=0$, $\sigma=0.5$
    ')
axes[1, 0].plot(x, stats.norm.pdf(x, 2, 1), label='$\mu=2$, $\sigma=1$')
axes[1, 0].legend()
axes[1, 0].set_title('Gaussian Distributions with Different Parameters')

# Poisson distribution
lambda_param = 3
```

```python
poisson_x = np.arange(0, 15)
poisson_pmf = stats.poisson.pmf(poisson_x, lambda_param)
axes[1, 1].bar(poisson_x, poisson_pmf)
axes[1, 1].set_title(f'Poisson Distribution ($\lambda={lambda_param}$)')
axes[1, 1].set_xlabel('k')
axes[1, 1].set_ylabel('Probability')

plt.tight_layout()
plt.show()
```

Listing 2.3: Probability Distributions

2.3.2 Statistical Measures

Central Tendency:

1. **Mean:** $\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$
2. **Median:** Middle value when sorted
3. **Mode:** Most frequent value

Dispersion:

1. **Variance:** $\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$
2. **Standard Deviation:** $\sigma = \sqrt{\sigma^2}$
3. **Covariance:** $\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$
4. **Correlation:** $\rho_{X,Y} = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y}$

Example

[Statistical Measures Implementation]

```python
# Generate sample data
np.random.seed(42)
data = np.random.normal(100, 15, 1000)

# Statistical measures
mean = np.mean(data)
median = np.median(data)
mode = stats.mode(data, keepdims=True)[0][0]
variance = np.var(data)
std_dev = np.std(data)

print(f"Mean: {mean:.2f}")
print(f"Median: {median:.2f}")
print(f"Mode: {mode:.2f}")
print(f"Variance: {variance:.2f}")
print(f"Standard Deviation: {std_dev:.2f}")

# Correlation example
x = np.random.normal(0, 1, 100)
y = 2 * x + np.random.normal(0, 0.5, 100)  # Correlated with noise

```

```python
correlation = np.corrcoef(x, y)[0, 1]
print(f"\nCorrelation between x and y: {correlation:.3f}")

# Visualize
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].hist(data, bins=30, edgecolor='black', alpha=0.7)
axes[0].axvline(mean, color='red', linestyle='--', label=f'Mean: {mean:.1f}')
axes[0].axvline(median, color='green', linestyle='--', label=f'Median: {median:.1f}')
axes[0].legend()
axes[0].set_title('Distribution with Central Tendency Measures')

axes[1].scatter(x, y, alpha=0.5)
axes[1].plot(np.unique(x), 2 * np.unique(x), 'r--', label='True relationship')
axes[1].set_xlabel('X')
axes[1].set_ylabel('Y')
axes[1].set_title(f'Correlation: {correlation:.3f}')
axes[1].legend()

plt.tight_layout()
plt.show()
```

Listing 2.4: Statistical Analysis

2.4 Calculus for Machine Learning

2.4.1 Derivatives and Gradients

Definition

The **derivative** measures the rate of change of a function.

For $f : \mathbb{R} \to \mathbb{R}$:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Partial Derivatives:

For functions of multiple variables $f : \mathbb{R}^n \to \mathbb{R}$:

$$\frac{\partial f}{\partial x_i} = \lim_{h \to 0} \frac{f(x_1, ..., x_i + h, ..., x_n) - f(x_1, ..., x_i, ..., x_n)}{h}$$

Gradient:

Vector of all partial derivatives:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

2.4.2 Chain Rule

For composite functions $f(g(x))$:

$$\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x)$$

This is fundamental for backpropagation in neural networks.

2.4.3 Optimization with Gradient Descent

Gradient Descent Algorithm:

1. Initialize parameters θ
2. Repeat until convergence:
$$\theta := \theta - \alpha \nabla_\theta J(\theta)$$
where α is the learning rate and $J(\theta)$ is the cost function.

Example

[Gradient Descent Implementation]

```
grad = gradient (x) x = x - learning_rate * grad
        x_history.append(x)
        f_history.append(f(x))

    # Visualization
    fig, axes = plt.subplots(1, 2, figsize=(14, 6))

    # Plot 1: axes[0].plot(3, 5, 'g *', markersize=15, label='Global
    minimum') axes[0].set_xlabel('x') axes[0].set_ylabel('f(x)') axes[0].
    set_title(Gradient Descent Optimization') axes[0].legend() axes[0].grid
    (True, alpha=0.3) # Plot 2: Convergence over iterations
    axes[1].plot(range(len(f_history)), f_history, 'b-')
    axes[1].set_xlabel('Iteration')
    axes[1].set_ylabel('Function Value')
    axes[1].set_title('Convergence Plot')
    axes[1].axhline(y=5, color='r', linestyle='--', alpha=0.5, label='True
     minimum')
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    print(f"Final x: {x_history[-1]:.6f}")
    print(f"Final f(x): {f_history[-1]:.6f}")
    print(f"True minimum: x=3, f(x)=5")

gradient_descent_demo()
```

Listing 2.5: Gradient Descent Demo

2.4.4 Multivariable Optimization

For functions of multiple variables, we use the gradient vector:

Example

[Multivariate Gradient Descent]

```python
def multivariate_gradient_descent():
    """
    Minimize f(x, y) = x   + y   + 2x - 4y + 5
    Minimum at x = -1, y = 2, f(-1, 2) = 0
    """

    def f(x, y):
        return x**2 + y**2 + 2*x - 4*y + 5

    def gradient(x, y):
        df_dx = 2*x + 2
        df_dy = 2*y - 4
        return np.array([df_dx, df_dy])

    # Initialize
    point = np.array([3.0, -1.0])
    learning_rate = 0.1
    iterations = 50

    # History
    history = [point.copy()]

    # Optimization
    for _ in range(iterations):
        grad = gradient(point[0], point[1])
        point = point - learning_rate * grad
        history.append(point.copy())

    history = np.array(history)

    # Visualization
    fig = plt.figure(figsize=(12, 5))

    # 3D surface plot
    ax1 = fig.add_subplot(121, projection='3d')
    x_range = np.linspace(-4, 4, 50)
    y_range = np.linspace(-4, 6, 50)
    X, Y = np.meshgrid(x_range, y_range)
    Z = f(X, Y)

    ax1.plot_surface(X, Y, Z, alpha=0.6, cmap='viridis')
    ax1.plot(history[:, 0], history[:, 1],
             [f(p[0], p[1]) for p in history],
             'r.-', markersize=8, linewidth=2)
    ax1.set_xlabel('x')
    ax1.set_ylabel('y')
    ax1.set_zlabel('f(x, y)')
    ax1.set_title('3D Optimization Path')

    # Contour plot
    ax2 = fig.add_subplot(122)
    contour = ax2.contour(X, Y, Z, levels=20, cmap='viridis')
    ax2.clabel(contour, inline=True, fontsize=8)
    ax2.plot(history[:, 0], history[:, 1], 'r.-', markersize=8, linewidth
    =2)
    ax2.plot(-1, 2, 'g*', markersize=15, label='True minimum')
    ax2.set_xlabel('x')
```

```python
    ax2.set_ylabel('y')
    ax2.set_title('Contour Plot of Optimization')
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    print(f"Final point: ({history[-1, 0]:.4f}, {history[-1, 1]:.4f})")
    print(f"Final value: {f(history[-1, 0], history[-1, 1]):.4f}")
    print(f"True minimum: (-1, 2) with value 0")

multivariate_gradient_descent()
```

Listing 2.6: Multivariate Optimization

2.5 Information Theory for ML

2.5.1 Entropy

Entropy measures the uncertainty in a random variable:

$$H(X) = -\sum_i p(x_i) \log p(x_i)$$

2.5.2 Cross-Entropy

Used as a loss function in classification:

$$H(p, q) = -\sum_i p(x_i) \log q(x_i)$$

2.5.3 KL Divergence

Measures difference between two probability distributions:

$$D_{KL}(P||Q) = \sum_i p(x_i) \log \frac{p(x_i)}{q(x_i)}$$

Example

[Information Theory Measures]

```python
def information_theory_demo():
    """Demonstrate entropy, cross-entropy, and KL divergence"""

    # Example distributions
    p = np.array([0.5, 0.3, 0.2])  # True distribution
    q = np.array([0.6, 0.3, 0.1])  # Predicted distribution

    # Entropy
    entropy_p = -np.sum(p * np.log2(p))
    print(f"Entropy of p: {entropy_p:.3f} bits")

    # Cross-entropy
```

```python
    cross_entropy = -np.sum(p * np.log2(q))
    print(f"Cross-entropy H(p, q): {cross_entropy:.3f} bits")

    # KL Divergence
    kl_divergence = np.sum(p * np.log2(p / q))
    print(f"KL Divergence D(p||q): {kl_divergence:.3f} bits")

    # Visualization
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

    x = np.arange(len(p))
    width = 0.35

    # Distribution comparison
    axes[0].bar(x - width/2, p, width, label='True (p)', alpha=0.8)
    axes[0].bar(x + width/2, q, width, label='Predicted (q)', alpha=0.8)
    axes[0].set_xlabel('Event')
    axes[0].set_ylabel('Probability')
    axes[0].set_title('Probability Distributions')
    axes[0].legend()
    axes[0].set_xticks(x)

    # Entropy visualization for different distributions
    probs_range = np.linspace(0.01, 0.99, 100)
    binary_entropy = -probs_range * np.log2(probs_range) - (1 -
   probs_range) * np.log2(1 - probs_range)

    axes[1].plot(probs_range, binary_entropy)
    axes[1].set_xlabel('Probability of Event 1')
    axes[1].set_ylabel('Entropy (bits)')
    axes[1].set_title('Binary Entropy Function')
    axes[1].grid(True, alpha=0.3)
    axes[1].axvline(x=0.5, color='r', linestyle='--', alpha=0.5, label='
   Maximum entropy')
    axes[1].legend()

    # Information measures comparison
    measures = ['Entropy(p)', 'Cross-Entropy(p,q)', 'KL-Divergence(p||q)']
    values = [entropy_p, cross_entropy, kl_divergence]
    colors = ['blue', 'orange', 'green']

    axes[2].bar(measures, values, color=colors, alpha=0.7)
    axes[2].set_ylabel('Bits')
    axes[2].set_title('Information Measures Comparison')
    axes[2].grid(True, alpha=0.3, axis='y')

    # Add value labels on bars
    for i, (measure, value) in enumerate(zip(measures, values)):
        axes[2].text(i, value + 0.02, f'{value:.3f}', ha='center')

    plt.tight_layout()
    plt.show()

information_theory_demo()
```

Listing 2.7: Information Theory Demo

2.6 Chapter Summary

In this chapter, we covered:

- **Linear Algebra:** Vectors, matrices, and operations essential for ML
- **Probability & Statistics:** Distributions, statistical measures, and Bayes' theorem
- **Calculus:** Derivatives, gradients, and optimization techniques
- **Information Theory:** Entropy and divergence measures

These mathematical foundations are crucial for understanding how machine learning algorithms work.

2.7 Exercises

2.7.1 Mathematical Exercises

1. **Linear Algebra:**
   - Prove that $(AB)^T = B^T A^T$
   - Show that the eigenvalues of a symmetric matrix are real
   - Implement matrix multiplication without using `np.dot()`
2. **Probability:**
   - Derive Bayes' theorem from the product rule
   - Calculate the entropy of a fair six-sided die
   - Prove that KL divergence is always non-negative
3. **Calculus:**
   - Find the minimum of $f(x, y) = x^2 + 2y^2 - 2xy + 4x - 6y$
   - Implement Newton's method for optimization
   - Derive the gradient descent update rule for linear regression

2.7.2 Programming Exercises

1. **Implement PCA from Scratch:**

```python
def pca_from_scratch(X, n_components):
    """
    Implement Principal Component Analysis
    Args:
        X: Data matrix (n_samples, n_features)
        n_components: Number of components to keep
    Returns:
        transformed_data: Data in PC space
        components: Principal components
    """
    # Your implementation here
    pass

```

Listing 2.8: PCA Implementation

2. **Gradient Descent Variants:**

```python
def sgd_vs_batch_gd(X, y, learning_rate=0.01, iterations=100):
    """
    Compare Stochastic Gradient Descent with Batch Gradient Descent
    for linear regression
    """
    # Your implementation here
    pass

```

Listing 2.9: Gradient Descent Comparison

3. **Statistical Hypothesis Testing:**

```python
def ab_test_simulator(n_samples=1000, effect_size=0.1):
    """
    Simulate an A/B test and calculate p-value
    """
    # Your implementation here
    pass

```

Listing 2.10: A/B Testing Simulation

Chapter 3

Data Preprocessing and Feature Engineering

3.1 Learning Objectives

Learning Objectives

By the end of this chapter, you will:

- Master data cleaning and preprocessing techniques
- Handle missing data, outliers, and imbalanced datasets
- Perform feature scaling and transformation
- Create meaningful features through feature engineering
- Select the most relevant features for your models

3.2 Data Quality and Cleaning

3.2.1 Understanding Data Quality Issues

Definition

Data quality problems can significantly impact model performance. Common issues include:

1. **Missing Data:** Values that are absent
2. **Outliers:** Extreme values that deviate from the pattern
3. **Duplicates:** Repeated records
4. **Inconsistencies:** Different formats or representations
5. **Errors:** Incorrect values due to measurement or entry errors

Example

[Creating Sample Data with Quality Issues]

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_classification

# Create a sample dataset with various issues
np.random.seed(42)

# Generate base data
n_samples = 1000
data = {
    'age': np.random.normal(35, 12, n_samples),
    'income': np.random.lognormal(10.5, 1.2, n_samples),
    'credit_score': np.random.normal(650, 100, n_samples),
    'years_employed': np.random.exponential(5, n_samples),
    'loan_amount': np.random.normal(15000, 5000, n_samples)
}

df = pd.DataFrame(data)

# Introduce data quality issues
# 1. Missing values
missing_indices = np.random.choice(df.index, size=100, replace=False)
df.loc[missing_indices, 'income'] = np.nan

missing_indices = np.random.choice(df.index, size=50, replace=False)
df.loc[missing_indices, 'credit_score'] = np.nan

# 2. Outliers
outlier_indices = np.random.choice(df.index, size=20, replace=False)
df.loc[outlier_indices, 'age'] = np.random.uniform(80, 120, size=len(
    outlier_indices))

# 3. Negative values (errors)
error_indices = np.random.choice(df.index, size=30, replace=False)
df.loc[error_indices, 'years_employed'] = -np.abs(df.loc[error_indices, '
    years_employed'])

# 4. Duplicates
duplicate_indices = np.random.choice(df.index, size=10, replace=False)
df = pd.concat([df, df.loc[duplicate_indices]], ignore_index=True)

print("Dataset with quality issues:")
print(f"Shape: {df.shape}")
print(f"\nMissing values:\n{df.isnull().sum()}")
print(f"\nDuplicates: {df.duplicated().sum()}")
print(f"\nNegative years_employed: {(df['years_employed'] < 0).sum()}")
```

Listing 3.1: Data Quality Issues Simulation

3.2.2 Data Profiling and Exploration

Example

[Comprehensive Data Profiling]

```python
def comprehensive_data_profile(df):
    """
    Generate comprehensive data profile report
    """
    print("=" * 60)
    print("DATA PROFILING REPORT")
    print("=" * 60)

    # Basic information
    print("\n1. BASIC INFORMATION")
    print(f"Shape: {df.shape}")
    print(f"Memory usage: {df.memory_usage(deep=True).sum() / 1024**2:.2f} MB")

    # Data types
    print("\n2. DATA TYPES")
    print(df.dtypes.value_counts())

    # Missing values
    print("\n3. MISSING VALUES")
    missing_df = pd.DataFrame({
        'Column': df.columns,
        'Missing_Count': df.isnull().sum(),
        'Missing_Percentage': (df.isnull().sum() / len(df)) * 100
    })
    print(missing_df[missing_df['Missing_Count'] > 0].sort_values('Missing_Percentage', ascending=False))

    # Duplicates
    print(f"\n4. DUPLICATES")
    print(f"Duplicate rows: {df.duplicated().sum()} ({(df.duplicated().sum() / len(df)) * 100:.2f}%)")

    # Numerical statistics
    print("\n5. NUMERICAL STATISTICS")
    print(df.describe())

    # Outlier detection using IQR
    print("\n6. OUTLIER DETECTION (IQR Method)")
    for col in df.select_dtypes(include=[np.number]).columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)][col].dropna()
        if len(outliers) > 0:
            print(f"{col}: {len(outliers)} outliers ({(len(outliers) / len(df[col].dropna())) * 100:.2f}%)")

    # Cardinality for categorical variables
    categorical_cols = df.select_dtypes(include=['object']).columns
    if len(categorical_cols) > 0:
        print("\n7. CATEGORICAL VARIABLES CARDINALITY")
```

```python
        for col in categorical_cols:
            print(f"{col}: {df[col].nunique()} unique values")

    # Correlation analysis
    print("\n8. CORRELATION ANALYSIS")
    numeric_cols = df.select_dtypes(include=[np.number]).columns
    if len(numeric_cols) > 1:
        corr_matrix = df[numeric_cols].corr()
        high_corr = []
        for i in range(len(corr_matrix.columns)):
            for j in range(i+1, len(corr_matrix.columns)):
                if abs(corr_matrix.iloc[i, j]) > 0.7:
                    high_corr.append((corr_matrix.columns[i],
                                      corr_matrix.columns[j],
                                      corr_matrix.iloc[i, j]))

        if high_corr:
            print("Highly correlated features (|r| > 0.7):")
            for feat1, feat2, corr in high_corr:
                print(f"  {feat1} - {feat2}: {corr:.3f}")
        else:
            print("No highly correlated features found")

# Run profiling
comprehensive_data_profile(df)

# Visualization of data issues
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

# Missing data heatmap
axes[0, 0].set_title('Missing Data Pattern')
missing_mask = df.isnull()
sns.heatmap(missing_mask, cbar=True, ax=axes[0, 0], cmap='RdYlBu',
            yticklabels=False, xticklabels=True)

# Distribution of age with outliers
axes[0, 1].set_title('Age Distribution (with outliers)')
axes[0, 1].hist(df['age'].dropna(), bins=50, edgecolor='black', alpha=0.7)
axes[0, 1].axvline(df['age'].mean(), color='red', linestyle='--', label='Mean')
axes[0, 1].legend()

# Box plots for outlier detection
axes[0, 2].set_title('Box Plots for Outlier Detection')
df_normalized = df.select_dtypes(include=[np.number]).dropna()
df_normalized = (df_normalized - df_normalized.mean()) / df_normalized.std()
axes[0, 2].boxplot(df_normalized.values, labels=df_normalized.columns)
axes[0, 2].tick_params(axis='x', rotation=45)

# Income distribution (with missing values noted)
axes[1, 0].set_title(f'Income Distribution ({df["income"].isnull().sum()} missing)')
axes[1, 0].hist(df['income'].dropna(), bins=50, edgecolor='black', alpha=0.7)
axes[1, 0].set_xlabel('Income')
axes[1, 0].set_ylabel('Frequency')

# Correlation heatmap
axes[1, 1].set_title('Feature Correlation Matrix')
```

```python
corr_matrix = df.select_dtypes(include=[np.number]).corr()
sns.heatmap(corr_matrix, annot=True, fmt='.2f', ax=axes[1, 1], cmap='coolwarm', center=0)

# Data quality summary
axes[1, 2].axis('off')
quality_summary = f"""
Data Quality Summary:
    Total records: {len(df)}
    Duplicate records: {df.duplicated().sum()}
    Columns with missing data: {(df.isnull().sum() > 0).sum()}
    Total missing values: {df.isnull().sum().sum()}
    Negative years_employed: {(df['years_employed'] < 0).sum()}
"""
axes[1, 2].text(0.1, 0.5, quality_summary, fontsize=12, verticalalignment='center')
axes[1, 2].set_title('Quality Issues Summary')

plt.tight_layout()
plt.show()
```

Listing 3.2: Data Profiling Function

3.2.3 Handling Missing Data

Example

[Missing Data Handling Strategies]

```python
class MissingDataHandler:
    """
    Comprehensive missing data handling strategies
    """

    def __init__(self, df):
        self.df = df.copy()
        self.original_shape = df.shape

    def analyze_missing_pattern(self):
        """Analyze patterns in missing data"""

        # Missing data statistics
        missing_stats = pd.DataFrame({
            'Column': self.df.columns,
            'Missing_Count': self.df.isnull().sum(),
            'Missing_Percentage': (self.df.isnull().sum() / len(self.df)) * 100,
            'Data_Type': self.df.dtypes
        })

        # Classify missingness
        missing_stats['Pattern'] = missing_stats['Missing_Percentage'].apply(
            lambda x: 'No Missing' if x == 0
            else 'Low (<5%)' if x < 5
            else 'Medium (5-20%)' if x < 20
            else 'High (20-50%)' if x < 50
```

```
            else 'Very High (>50%)'
        )

        return missing_stats[missing_stats['Missing_Count'] > 0].sort_values('Missing_Percentage', ascending=False)

    def deletion_strategies(self):
        """Different deletion strategies"""

        strategies = {}

        # 1. Listwise deletion (remove rows with any missing)
        strategies['listwise'] = self.df.dropna()

        # 2. Pairwise deletion (remove rows with missing in specific columns)
        important_cols = ['age', 'income']  # Define important columns
        strategies['pairwise'] = self.df.dropna(subset=important_cols)

        # 3. Column deletion (remove columns with >50% missing)
        threshold = 0.5
        missing_prop = self.df.isnull().sum() / len(self.df)
        cols_to_keep = missing_prop[missing_prop < threshold].index
        strategies['column_deletion'] = self.df[cols_to_keep]

        return strategies

    def imputation_strategies(self):
        """Different imputation strategies"""

        from sklearn.impute import SimpleImputer, KNNImputer
        from sklearn.experimental import enable_iterative_imputer
        from sklearn.impute import IterativeImputer

        strategies = {}
        df_numeric = self.df.select_dtypes(include=[np.number])

        # 1. Mean imputation
        mean_imputer = SimpleImputer(strategy='mean')
        strategies['mean'] = pd.DataFrame(
            mean_imputer.fit_transform(df_numeric),
            columns=df_numeric.columns,
            index=df_numeric.index
        )

        # 2. Median imputation
        median_imputer = SimpleImputer(strategy='median')
        strategies['median'] = pd.DataFrame(
            median_imputer.fit_transform(df_numeric),
            columns=df_numeric.columns,
            index=df_numeric.index
        )

        # 3. Mode imputation
        mode_imputer = SimpleImputer(strategy='most_frequent')
        strategies['mode'] = pd.DataFrame(
            mode_imputer.fit_transform(df_numeric),
            columns=df_numeric.columns,
            index=df_numeric.index
        )
```

```python

        # 4. Forward/Backward fill (for time series)
        strategies['forward_fill'] = df_numeric.fillna(method='ffill')
        strategies['backward_fill'] = df_numeric.fillna(method='bfill')

        # 5. KNN imputation
        knn_imputer = KNNImputer(n_neighbors=5)
        strategies['knn'] = pd.DataFrame(
            knn_imputer.fit_transform(df_numeric),
            columns=df_numeric.columns,
            index=df_numeric.index
        )

        # 6. Iterative imputation (MICE)
        iterative_imputer = IterativeImputer(random_state=42)
        strategies['iterative'] = pd.DataFrame(
            iterative_imputer.fit_transform(df_numeric),
            columns=df_numeric.columns,
            index=df_numeric.index
        )

        return strategies

# Apply missing data handling
handler = MissingDataHandler(df)

print("Missing Data Pattern Analysis:")
print(handler.analyze_missing_pattern())

print("\n" + "="*60)
print("Imputation Strategies Comparison:")

# Get imputation strategies
imputation_strategies = handler.imputation_strategies()

# Visualize imputation results
fig, axes = plt.subplots(2, 4, figsize=(16, 8))
axes = axes.ravel()

for idx, (name, imputed_df) in enumerate(imputation_strategies.items()):
    if idx < len(axes):
        # Plot distribution of imputed income values
        original_income = df['income'].dropna()
        imputed_income = imputed_df['income']

        axes[idx].hist(original_income, bins=30, alpha=0.5, label='Original', density=True)
        axes[idx].hist(imputed_income, bins=30, alpha=0.5, label='Imputed', density=True)
        axes[idx].set_title(f'{name.capitalize()} Imputation')
        axes[idx].set_xlabel('Income')
        axes[idx].set_ylabel('Density')
        axes[idx].legend()

plt.tight_layout()
plt.show()
```

Listing 3.3: Comprehensive Missing Data Handler

3.2.4 Outlier Detection and Treatment

Example

[Outlier Detection and Treatment]

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
from scipy import stats

class OutlierHandler:
    """
    Comprehensive outlier detection and treatment
    """

    def __init__(self, df):
        self.df = df.copy()

    def detect_outliers_iqr(self, column, k=1.5):
        """Detect outliers using IQR method"""
        Q1 = self.df[column].quantile(0.25)
        Q3 = self.df[column].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - k * IQR
        upper_bound = Q3 + k * IQR

        outliers_mask = (self.df[column] < lower_bound) | (self.df[column]
     > upper_bound)
        return outliers_mask, lower_bound, upper_bound

    def detect_outliers_zscore(self, column, threshold=3):
        """Detect outliers using Z-score method"""
        z_scores = np.abs(stats.zscore(self.df[column].dropna()))
        outliers_mask = z_scores > threshold

        # Align mask with original dataframe
        full_mask = pd.Series(False, index=self.df.index)
        full_mask[self.df[column].notna()] = outliers_mask

        return full_mask

    def detect_outliers_isolation_forest(self, columns):
        """Detect outliers using Isolation Forest"""
        # Prepare data
        X = self.df[columns].dropna()

        # Fit Isolation Forest
        iso_forest = IsolationForest(contamination=0.1, random_state=42)
        outliers = iso_forest.fit_predict(X)

        # Create mask (-1 indicates outlier)
        outliers_mask = pd.Series(False, index=self.df.index)
        outliers_mask[X.index] = (outliers == -1)

        return outliers_mask

```

```python
    def detect_outliers_dbscan(self, columns, eps=0.5, min_samples=5):
        """Detect outliers using DBSCAN clustering"""
        # Prepare and scale data
        X = self.df[columns].dropna()
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

        # Apply DBSCAN
        dbscan = DBSCAN(eps=eps, min_samples=min_samples)
        clusters = dbscan.fit_predict(X_scaled)

        # Outliers are labeled as -1
        outliers_mask = pd.Series(False, index=self.df.index)
        outliers_mask[X.index] = (clusters == -1)

        return outliers_mask

    def treat_outliers(self, column, method='cap'):
        """Different outlier treatment methods"""
        treated_series = self.df[column].copy()
        outliers_mask, lower_bound, upper_bound = self.detect_outliers_iqr(column)

        if method == 'remove':
            # Remove outliers
            treated_series[outliers_mask] = np.nan

        elif method == 'cap':
            # Cap outliers at bounds
            treated_series[self.df[column] < lower_bound] = lower_bound
            treated_series[self.df[column] > upper_bound] = upper_bound

        elif method == 'transform':
            # Log transformation
            treated_series = np.log1p(treated_series)

        elif method == 'winsorize':
            # Winsorize at 5% and 95% percentiles
            from scipy.stats import mstats
            treated_series = pd.Series(
                mstats.winsorize(treated_series.dropna(), limits=(0.05, 0.05)),
                index=treated_series.dropna().index
            )

        return treated_series

    def visualize_outliers(self, column):
        """Visualize outliers using multiple methods"""
        fig, axes = plt.subplots(2, 3, figsize=(15, 10))

        # Original distribution
        axes[0, 0].hist(self.df[column].dropna(), bins=50, edgecolor='black')
        axes[0, 0].set_title('Original Distribution')
        axes[0, 0].set_xlabel(column)

        # Box plot
        axes[0, 1].boxplot(self.df[column].dropna())
        axes[0, 1].set_title('Box Plot')
```

```
        axes[0, 1].set_ylabel(column)

        # IQR method
        outliers_iqr, lower, upper = self.detect_outliers_iqr(column)
        axes[0, 2].scatter(range(len(self.df)), self.df[column], alpha=0.5)
        axes[0, 2].scatter(np.where(outliers_iqr)[0],
                           self.df[column][outliers_iqr],
                           color='red', label='Outliers')
        axes[0, 2].axhline(y=lower, color='r', linestyle='--', label=f'Lower: {lower:.2f}')
        axes[0, 2].axhline(y=upper, color='r', linestyle='--', label=f'Upper: {upper:.2f}')
        axes[0, 2].set_title('IQR Method')
        axes[0, 2].legend()

        # Z-score method
        outliers_z = self.detect_outliers_zscore(column)
        axes[1, 0].scatter(range(len(self.df)), self.df[column], alpha=0.5)
        axes[1, 0].scatter(np.where(outliers_z)[0],
                           self.df[column][outliers_z],
                           color='red', label='Outliers')
        axes[1, 0].set_title('Z-Score Method')
        axes[1, 0].legend()

        # After capping
        treated_cap = self.treat_outliers(column, method='cap')
        axes[1, 1].hist(treated_cap.dropna(), bins=50, edgecolor='black', alpha=0.7, color='green')
        axes[1, 1].set_title('After Capping')
        axes[1, 1].set_xlabel(column)

        # After log transformation
        treated_log = self.treat_outliers(column, method='transform')
        axes[1, 2].hist(treated_log.dropna(), bins=50, edgecolor='black', alpha=0.7, color='blue')
        axes[1, 2].set_title('After Log Transform')
        axes[1, 2].set_xlabel(f'log({column})')

        plt.tight_layout()
        plt.show()

        # Print summary
        print(f"Outlier Detection Summary for '{column}':")
        print(f"IQR Method: {outliers_iqr.sum()} outliers ({outliers_iqr.sum()/len(self.df)*100:.2f}%)")
        print(f"Z-Score Method: {outliers_z.sum()} outliers ({outliers_z.sum()/len(self.df)*100:.2f}%)")

# Example usage
if __name__ == "__main__":
    # Create sample data
    np.random.seed(42)
    data = {
        'age': np.random.normal(35, 12, 1000),
        'income': np.random.lognormal(10.5, 1.2, 1000),
    }
    df = pd.DataFrame(data)

```

```
    # Add some outliers
    outlier_indices = np.random.choice(df.index, size=20, replace=False)
    df.loc[outlier_indices, 'age'] = np.random.uniform(80, 120, size=len(
    outlier_indices))

    handler = OutlierHandler(df)
    handler.visualize_outliers('age')
```

Listing 3.4: Comprehensive Outlier Handler

3.3 Feature Scaling and Transformation

3.3.1 Feature Scaling Techniques

Example

[Feature Scaling Comparison]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler, MinMaxScaler,
    RobustScaler, MaxAbsScaler, Normalizer, QuantileTransformer

class FeatureScaler:
    """
    Comprehensive feature scaling techniques
    """

    def __init__(self, X):
        self.X = X
        self.scalers = {
            'original': None,
            'standard': StandardScaler(),
            'minmax': MinMaxScaler(),
            'robust': RobustScaler(),
            'maxabs': MaxAbsScaler(),
            'l2_norm': Normalizer(norm='l2'),
            'quantile_uniform': QuantileTransformer(output_distribution='
    uniform'),
            'quantile_normal': QuantileTransformer(output_distribution='
    normal')
        }
        self.scaled_data = {}

    def apply_all_scalers(self):
        """Apply all scaling techniques"""
        self.scaled_data['original'] = self.X

        for name, scaler in self.scalers.items():
            if name != 'original':
                self.scaled_data[name] = scaler.fit_transform(self.X) if
    scaler else self.X

        return self.scaled_data

```

```python
    def visualize_scaling_effects(self):
        """Visualize the effect of different scaling techniques"""
        n_features = self.X.shape[1]
        n_scalers = len(self.scaled_data)

        fig, axes = plt.subplots(n_scalers, n_features + 1,
                                 figsize=(15, 2.5 * n_scalers))

        for i, (name, data) in enumerate(self.scaled_data.items()):
            # Distribution plots for each feature
            for j in range(n_features):
                axes[i, j].hist(data[:, j], bins=30, alpha=0.7, edgecolor='black')
                axes[i, j].set_title(f'{name}\nFeature {j}')
                axes[i, j].set_xlabel('Value')
                axes[i, j].set_ylabel('Frequency')

                # Add statistics
                mean = np.mean(data[:, j])
                std = np.std(data[:, j])
                axes[i, j].axvline(mean, color='r', linestyle='--', alpha=0.5)
                axes[i, j].text(0.7, 0.9, f'  ={mean:.2f}\n  ={std:.2f}',
                                transform=axes[i, j].transAxes)

            # Box plot for all features
            axes[i, -1].boxplot(data)
            axes[i, -1].set_title(f'{name}\nAll Features')
            axes[i, -1].set_xlabel('Feature')
            axes[i, -1].set_ylabel('Value')

        plt.tight_layout()
        plt.show()

    def compare_scalers_metrics(self):
        """Compare statistical metrics across scaling methods"""
        metrics_df = pd.DataFrame()

        for name, data in self.scaled_data.items():
            for i in range(data.shape[1]):
                metrics_df = pd.concat([metrics_df, pd.DataFrame({
                    'Scaler': [name],
                    'Feature': [f'Feature_{i}'],
                    'Mean': [np.mean(data[:, i])],
                    'Std': [np.std(data[:, i])],
                    'Min': [np.min(data[:, i])],
                    'Max': [np.max(data[:, i])],
                    'Range': [np.max(data[:, i]) - np.min(data[:, i])]
                })], ignore_index=True)

        return metrics_df

# Example usage
if __name__ == "__main__":
    # Create sample data with different scales
    np.random.seed(42)
    X_scaling = np.column_stack([
        np.random.normal(100, 15, 1000),      # Feature 1: Normal(100, 15)
        np.random.exponential(2, 1000) * 10,   # Feature 2: Exponential, scaled
```

```
        np.random.uniform(0, 1, 1000),          # Feature 3: Uniform(0, 1)
        np.random.lognormal(3, 1, 1000)         # Feature 4: Lognormal
    ])

    # Apply scaling
    scaler = FeatureScaler(X_scaling)
    scaled_data = scaler.apply_all_scalers()

    # Visualize effects
    scaler.visualize_scaling_effects()

    # Compare metrics
    print("Scaling Methods Comparison:")
    print(scaler.compare_scalers_metrics().pivot_table(
        values=['Mean', 'Std', 'Range'],
        index='Feature',
        columns='Scaler'
    ))
```

Listing 3.5: Comprehensive Feature Scaling

3.3.2 Feature Transformation Techniques

Example

[Feature Transformation Methods]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import PowerTransformer, FunctionTransformer,
    PolynomialFeatures, KBinsDiscretizer
from scipy import stats

class FeatureTransformer:
    """
    Feature transformation techniques
    """

    def __init__(self, X, feature_names=None):
        self.X = X
        self.feature_names = feature_names or [f'Feature_{i}' for i in
    range(X.shape[1])]
        self.transformed_data = {}

    def apply_transformations(self):
        """Apply various transformation techniques"""
        # Original
        self.transformed_data['original'] = self.X

        # Log transformation (add 1 to handle zeros)
        self.transformed_data['log'] = np.log1p(np.abs(self.X))

        # Square root transformation
        self.transformed_data['sqrt'] = np.sqrt(np.abs(self.X))

        # Square transformation
```

```python
        self.transformed_data['square'] = self.X ** 2

        # Reciprocal transformation (avoid division by zero)
        self.transformed_data['reciprocal'] = 1 / (self.X + 1e-8)

        # Box-Cox transformation (requires positive values)
        box_cox_data = np.zeros_like(self.X)
        for i in range(self.X.shape[1]):
            if np.all(self.X[:, i] > 0):
                box_cox_data[:, i], _ = stats.boxcox(self.X[:, i])
            else:
                # Shift data to make positive
                shifted = self.X[:, i] - np.min(self.X[:, i]) + 1
                box_cox_data[:, i], _ = stats.boxcox(shifted)
        self.transformed_data['box_cox'] = box_cox_data

        # Yeo-Johnson transformation (handles negative values)
        yeo_johnson = PowerTransformer(method='yeo-johnson')
        self.transformed_data['yeo_johnson'] = yeo_johnson.fit_transform(self.X)

        return self.transformed_data

    def test_normality(self):
        """Test normality of transformed features"""
        normality_results = []

        for name, data in self.transformed_data.items():
            for i in range(data.shape[1]):
                # Shapiro-Wilk test
                stat_shapiro, p_shapiro = stats.shapiro(data[:min(5000, len(data)), i])

                # D'Agostino's K-squared test
                stat_dagostino, p_dagostino = stats.normaltest(data[:, i])

                normality_results.append({
                    'Transformation': name,
                    'Feature': self.feature_names[i],
                    'Shapiro_Statistic': stat_shapiro,
                    'Shapiro_p_value': p_shapiro,
                    'Shapiro_Normal': 'Yes' if p_shapiro > 0.05 else 'No',
                    'DAgostino_Statistic': stat_dagostino,
                    'DAgostino_p_value': p_dagostino,
                    'DAgostino_Normal': 'Yes' if p_dagostino > 0.05 else 'No'
                })

        return pd.DataFrame(normality_results)

    def visualize_transformations(self, feature_idx=0):
        """Visualize transformation effects on a single feature"""
        fig, axes = plt.subplots(3, 3, figsize=(15, 12))
        axes = axes.ravel()

        for idx, (name, data) in enumerate(self.transformed_data.items()):
            if idx < len(axes):
                # Histogram
                axes[idx].hist(data[:, feature_idx], bins=50,
                               alpha=0.7, edgecolor='black', density=True)
```

```python

                # Fit and plot normal distribution
                mu, std = stats.norm.fit(data[:, feature_idx])
                xmin, xmax = axes[idx].get_xlim()
                x = np.linspace(xmin, xmax, 100)
                p = stats.norm.pdf(x, mu, std)
                axes[idx].plot(x, p, 'r-', linewidth=2, label='Normal fit')

                # Add Q-Q plot as inset
                inset_ax = axes[idx].inset_axes([0.55, 0.55, 0.4, 0.4])
                stats.probplot(data[:, feature_idx], dist="norm", plot=inset_ax)
                inset_ax.set_xlabel('')
                inset_ax.set_ylabel('')
                inset_ax.set_title('Q-Q', fontsize=8)
                inset_ax.tick_params(labelsize=6)

                # Title and labels
                axes[idx].set_title(f'{name.replace("_", " ").title()} Transformation')
                axes[idx].set_xlabel('Value')
                axes[idx].set_ylabel('Density')
                axes[idx].legend()

                # Add skewness and kurtosis
                skew = stats.skew(data[:, feature_idx])
                kurt = stats.kurtosis(data[:, feature_idx])
                axes[idx].text(0.02, 0.95, f'Skew: {skew:.2f}\nKurt: {kurt:.2f}',
                               transform=axes[idx].transAxes,
                               verticalalignment='top',
                               bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

        plt.suptitle(f'Transformation Effects on {self.feature_names[feature_idx]}',
                     fontsize=16)
        plt.tight_layout()
        plt.show()

# Example usage
if __name__ == "__main__":
    # Create skewed data for transformation
    np.random.seed(42)
    X_transform = np.column_stack([
        np.random.exponential(2, 1000),           # Right-skewed
        np.random.lognormal(0, 1, 1000),          # Right-skewed
        np.random.beta(2, 5, 1000),               # Left-skewed
        np.random.gamma(2, 2, 1000)               # Right-skewed
    ])

    feature_names = ['Exponential', 'Lognormal', 'Beta', 'Gamma']

    # Apply transformations
    transformer = FeatureTransformer(X_transform, feature_names)
    transformed_data = transformer.apply_transformations()

    # Visualize transformations for each feature
    for i in range(X_transform.shape[1]):
```

```
        transformer.visualize_transformations(feature_idx=i)

    # Test normality
    print("\nNormality Test Results:")
    normality_df = transformer.test_normality()
    print(normality_df[normality_df['Shapiro_Normal'] == 'Yes'][
        ['Transformation', 'Feature', 'Shapiro_p_value']
    ].sort_values('Shapiro_p_value', ascending=False))
```

Listing 3.6: Feature Transformation Techniques

3.4 Feature Engineering

3.4.1 Creating New Features

Example

[Comprehensive Feature Engineering]

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures, KBinsDiscretizer
from sklearn.ensemble import RandomForestRegressor

class FeatureEngineer:
    """
    Comprehensive feature engineering techniques
    """

    def __init__(self, df):
        self.df = df.copy()
        self.new_features = pd.DataFrame(index=df.index)

    def create_polynomial_features(self, columns, degree=2, include_bias=
    False):
        """Create polynomial and interaction features"""
        poly = PolynomialFeatures(degree=degree, include_bias=include_bias
    )
        poly_features = poly.fit_transform(self.df[columns])

        # Get feature names
        feature_names = poly.get_feature_names_out(columns)

        # Add to new features
        for i, name in enumerate(feature_names):
            if name not in columns:  # Don't include original features
                self.new_features[name] = poly_features[:, i]

        return self.new_features

    def create_binning_features(self, column, n_bins=5, strategy='quantile
    '):
        """Create binned features"""
        # Binning
        discretizer = KBinsDiscretizer(n_bins=n_bins, encode='ordinal',
```

```
strategy=strategy)
        binned = discretizer.fit_transform(self.df[[column]])
        self.new_features[f'{column}_binned'] = binned.ravel()

        # One-hot encoding of bins
        discretizer_onehot = KBinsDiscretizer(n_bins=n_bins, encode='onehot', strategy=strategy)
        binned_onehot = discretizer_onehot.fit_transform(self.df[[column]])

        for i in range(n_bins):
            self.new_features[f'{column}_bin_{i}'] = binned_onehot[:, i].toarray().ravel()

        return self.new_features

    def create_statistical_features(self, columns):
        """Create statistical aggregation features"""
        # Row-wise statistics
        subset = self.df[columns]

        self.new_features['row_mean'] = subset.mean(axis=1)
        self.new_features['row_std'] = subset.std(axis=1)
        self.new_features['row_min'] = subset.min(axis=1)
        self.new_features['row_max'] = subset.max(axis=1)
        self.new_features['row_range'] = subset.max(axis=1) - subset.min(axis=1)
        self.new_features['row_median'] = subset.median(axis=1)
        self.new_features['row_skew'] = subset.skew(axis=1)
        self.new_features['row_kurt'] = subset.kurtosis(axis=1)

        # Coefficient of variation
        self.new_features['row_cv'] = subset.std(axis=1) / (subset.mean(axis=1) + 1e-8)

        return self.new_features

    def create_ratio_features(self, numerator_cols, denominator_cols):
        """Create ratio features"""
        for num_col in numerator_cols:
            for den_col in denominator_cols:
                if num_col != den_col:
                    # Avoid division by zero
                    ratio = self.df[num_col] / (self.df[den_col] + 1e-8)
                    self.new_features[f'{num_col}_div_{den_col}'] = ratio

        return self.new_features

    def create_domain_specific_features(self):
        """Create domain-specific features (example for financial data)"""
        # Example: Financial ratios
        if 'income' in self.df.columns and 'loan_amount' in self.df.columns:
            self.new_features['debt_to_income'] = (
                self.df['loan_amount'] / (self.df['income'] + 1e-8)
            )

        if 'age' in self.df.columns:
            # Age groups
            self.new_features['age_group'] = pd.cut(
```

```
                self.df['age'],
                bins=[0, 25, 35, 45, 55, 65, 100],
                labels=['<25', '25-35', '35-45', '45-55', '55-65', '65+']
            )

            # One-hot encode age groups
            age_dummies = pd.get_dummies(self.new_features['age_group'], prefix='age_group')
            self.new_features = pd.concat([self.new_features, age_dummies], axis=1)
            self.new_features.drop('age_group', axis=1, inplace=True)

        if 'credit_score' in self.df.columns:
            # Credit score categories
            self.new_features['credit_category'] = pd.cut(
                self.df['credit_score'],
                bins=[0, 580, 670, 740, 800, 850],
                labels=['Poor', 'Fair', 'Good', 'Very Good', 'Excellent']
            )

            # Binary flags
            self.new_features['is_prime_borrower'] = (self.df['credit_score'] >= 740).astype(int)

        return self.new_features

    def create_temporal_features(self, date_column):
        """Create temporal features from date column"""
        if date_column in self.df.columns:
            dates = pd.to_datetime(self.df[date_column])

            # Date components
            self.new_features['year'] = dates.dt.year
            self.new_features['month'] = dates.dt.month
            self.new_features['day'] = dates.dt.day
            self.new_features['dayofweek'] = dates.dt.dayofweek
            self.new_features['quarter'] = dates.dt.quarter
            self.new_features['dayofyear'] = dates.dt.dayofyear
            self.new_features['weekofyear'] = dates.dt.isocalendar().week

            # Cyclical encoding
            self.new_features['month_sin'] = np.sin(2 * np.pi * dates.dt.month / 12)
            self.new_features['month_cos'] = np.cos(2 * np.pi * dates.dt.month / 12)
            self.new_features['day_sin'] = np.sin(2 * np.pi * dates.dt.day / 31)
            self.new_features['day_cos'] = np.cos(2 * np.pi * dates.dt.day / 31)

            # Binary flags
            self.new_features['is_weekend'] = (dates.dt.dayofweek >= 5).astype(int)
            self.new_features['is_month_start'] = dates.dt.is_month_start.astype(int)
            self.new_features['is_month_end'] = dates.dt.is_month_end.astype(int)

        return self.new_features

```

```python
# Example usage
if __name__ == "__main__":
    # Create sample data
    np.random.seed(42)
    data = {
        'age': np.random.normal(35, 12, 1000),
        'income': np.random.lognormal(10.5, 1.2, 1000),
        'credit_score': np.random.normal(650, 100, 1000),
        'years_employed': np.random.exponential(5, 1000),
        'loan_amount': np.random.normal(15000, 5000, 1000)
    }
    df = pd.DataFrame(data)

    # Apply feature engineering
    engineer = FeatureEngineer(df)

    # Create different types of features
    numeric_cols = ['age', 'income', 'credit_score', 'years_employed', 'loan_amount']

    # Polynomial features
    poly_features = engineer.create_polynomial_features(numeric_cols[:3], degree=2)
    print(f"Created {len(poly_features.columns)} polynomial features")

    # Statistical features
    stat_features = engineer.create_statistical_features(numeric_cols)
    print(f"Created {len(stat_features.columns)} statistical features")

    # Ratio features
    ratio_features = engineer.create_ratio_features(
        numerator_cols=['loan_amount'],
        denominator_cols=['income', 'years_employed']
    )

    # Domain-specific features
    domain_features = engineer.create_domain_specific_features()

    # Combine all features
    all_new_features = engineer.new_features
    print(f"\nTotal new features created: {len(all_new_features.columns)}")

    # Visualize feature importance (using Random Forest)
    # Create target variable (synthetic)
    y = df['loan_amount'].fillna(df['loan_amount'].mean())

    # Combine original and new features
    X_all = pd.concat([df[numeric_cols].fillna(df[numeric_cols].mean()),
                       all_new_features.select_dtypes(include=[np.number])], axis=1)

    # Train Random Forest to get feature importance
    rf = RandomForestRegressor(n_estimators=100, random_state=42, max_depth=5)
    rf.fit(X_all, y)

    # Get feature importance
    feature_importance = pd.DataFrame({
        'feature': X_all.columns,
```

```
        'importance': rf.feature_importances_
    }).sort_values('importance', ascending=False)

    # Visualize top features
    plt.figure(figsize=(12, 8))
    top_n = 20
    plt.barh(range(top_n), feature_importance.iloc[:top_n]['importance'])
    plt.yticks(range(top_n), feature_importance.iloc[:top_n]['feature'])
    plt.xlabel('Feature Importance')
    plt.title(f'Top {top_n} Most Important Features')
    plt.gca().invert_yaxis()
    plt.tight_layout()
    plt.show()

    print(f"\nTop 10 most important features:")
    print(feature_importance.head(10))
```

Listing 3.7: Feature Engineering Techniques

3.5 Feature Selection

3.5.1 Feature Selection Methods

Example

[Feature Selection Techniques]

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_selection import (SelectKBest, RFE, SelectFromModel,
                                       VarianceThreshold,
    mutual_info_regression, f_classif)
from sklearn.linear_model import LassoCV, Lasso
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from collections import Counter

class FeatureSelector:
    """
    Comprehensive feature selection techniques
    """

    def __init__(self, X, y, task='classification'):
        self.X = X
        self.y = y
        self.task = task
        self.selection_results = {}

    def variance_threshold(self, threshold=0.01):
        """Remove low variance features"""
        selector = VarianceThreshold(threshold=threshold)
        X_selected = selector.fit_transform(self.X)

        selected_features = self.X.columns[selector.get_support()]
        self.selection_results['variance'] = {
            'selected_features': selected_features,
```

```
            'n_features': len(selected_features),
            'selector': selector
        }

        return selected_features

    def univariate_selection(self, k=10):
        """Select k best features using statistical tests"""
        if self.task == 'classification':
            score_func = f_classif
        else:
            score_func = mutual_info_regression

        selector = SelectKBest(score_func=score_func, k=k)
        X_selected = selector.fit_transform(self.X, self.y)

        selected_features = self.X.columns[selector.get_support()]
        feature_scores = pd.DataFrame({
            'feature': self.X.columns,
            'score': selector.scores_
        }).sort_values('score', ascending=False)

        self.selection_results['univariate'] = {
            'selected_features': selected_features,
            'n_features': len(selected_features),
            'selector': selector,
            'scores': feature_scores
        }

        return selected_features

    def recursive_feature_elimination(self, estimator=None, n_features_to_select=10):
        """Recursive Feature Elimination"""
        if estimator is None:
            if self.task == 'classification':
                estimator = RandomForestClassifier(n_estimators=100, random_state=42)
            else:
                estimator = RandomForestRegressor(n_estimators=100, random_state=42)

        selector = RFE(estimator, n_features_to_select=n_features_to_select)
        X_selected = selector.fit_transform(self.X, self.y)

        selected_features = self.X.columns[selector.get_support()]
        ranking = pd.DataFrame({
            'feature': self.X.columns,
            'ranking': selector.ranking_
        }).sort_values('ranking')

        self.selection_results['rfe'] = {
            'selected_features': selected_features,
            'n_features': len(selected_features),
            'selector': selector,
            'ranking': ranking
        }

        return selected_features
```

```python

    def lasso_selection(self, alpha=None):
        """Feature selection using L1 regularization"""
        if alpha is None:
            lasso = LassoCV(cv=5, random_state=42)
            lasso.fit(self.X, self.y)
            alpha = lasso.alpha_
        else:
            lasso = Lasso(alpha=alpha)
            lasso.fit(self.X, self.y)

        # Select features with non-zero coefficients
        selected_features = self.X.columns[lasso.coef_ != 0]

        coefficients = pd.DataFrame({
            'feature': self.X.columns,
            'coefficient': lasso.coef_
        }).sort_values('coefficient', key=abs, ascending=False)

        self.selection_results['lasso'] = {
            'selected_features': selected_features,
            'n_features': len(selected_features),
            'selector': lasso,
            'coefficients': coefficients,
            'alpha': alpha
        }

        return selected_features

    def tree_based_selection(self, threshold='mean'):
        """Feature selection using tree-based models"""
        if self.task == 'classification':
            model = RandomForestClassifier(n_estimators=100, random_state
    =42)
        else:
            model = RandomForestRegressor(n_estimators=100, random_state
    =42)

        selector = SelectFromModel(model, threshold=threshold)
        selector.fit(self.X, self.y)

        selected_features = self.X.columns[selector.get_support()]

        # Get feature importances
        feature_importance = pd.DataFrame({
            'feature': self.X.columns,
            'importance': model.fit(self.X, self.y).feature_importances_
        }).sort_values('importance', ascending=False)

        self.selection_results['tree_based'] = {
            'selected_features': selected_features,
            'n_features': len(selected_features),
            'selector': selector,
            'importances': feature_importance
        }

        return selected_features

    def correlation_selection(self, threshold=0.9):
        """Remove highly correlated features"""
```

```python
        corr_matrix = self.X.corr().abs()
        upper_tri = corr_matrix.where(
            np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
        )

        # Find features to drop
        to_drop = [column for column in upper_tri.columns
                   if any(upper_tri[column] > threshold)]

        selected_features = self.X.columns.drop(to_drop)

        self.selection_results['correlation'] = {
            'selected_features': selected_features,
            'n_features': len(selected_features),
            'dropped_features': to_drop,
            'threshold': threshold
        }

        return selected_features

    def compare_methods(self):
        """Compare different feature selection methods"""
        comparison = pd.DataFrame()

        for method, results in self.selection_results.items():
            comparison = pd.concat([comparison, pd.DataFrame({
                'Method': [method],
                'N_Features_Selected': [results['n_features']],
                'Features': [', '.join(results['selected_features'][:5]) + '...']
            })], ignore_index=True)

        return comparison

    def visualize_selection(self):
        """Visualize feature selection results"""
        n_methods = len(self.selection_results)
        fig, axes = plt.subplots(2, 3, figsize=(15, 10))
        axes = axes.ravel()

        for idx, (method, results) in enumerate(self.selection_results.items()):
            if idx < len(axes):
                if method == 'univariate' and 'scores' in results:
                    # Plot feature scores
                    top_n = min(20, len(results['scores']))
                    axes[idx].barh(range(top_n),
                                   results['scores'].iloc[:top_n]['score'])
                    axes[idx].set_yticks(range(top_n))
                    axes[idx].set_yticklabels(
                        results['scores'].iloc[:top_n]['feature']
                    )
                    axes[idx].set_xlabel('Score')
                    axes[idx].invert_yaxis()

                elif method == 'lasso' and 'coefficients' in results:
                    # Plot coefficients
                    non_zero = results['coefficients'][
                        results['coefficients']['coefficient'] != 0
                    ]
```

```python
                    top_n = min(20, len(non_zero))
                    axes[idx].barh(range(top_n),
                                   non_zero.iloc[:top_n]['coefficient'])
                    axes[idx].set_yticks(range(top_n))
                    axes[idx].set_yticklabels(
                        non_zero.iloc[:top_n]['feature']
                    )
                    axes[idx].set_xlabel('Coefficient')
                    axes[idx].invert_yaxis()

                elif method == 'tree_based' and 'importances' in results:
                    # Plot feature importances
                    top_n = min(20, len(results['importances']))
                    axes[idx].barh(range(top_n),
                                   results['importances'].iloc[:top_n]['importance'])
                    axes[idx].set_yticks(range(top_n))
                    axes[idx].set_yticklabels(
                        results['importances'].iloc[:top_n]['feature']
                    )
                    axes[idx].set_xlabel('Importance')
                    axes[idx].invert_yaxis()

                else:
                    # Show selected features count
                    axes[idx].bar([method], [results['n_features']])
                    axes[idx].set_ylabel('Number of Features')

                axes[idx].set_title(f'{method.replace("_", " ").title()} Selection')

        plt.tight_layout()
        plt.show()

# Example usage
if __name__ == "__main__":
    from sklearn.datasets import make_classification

    # Create synthetic dataset
    X_data, y_data = make_classification(
        n_samples=1000,
        n_features=50,
        n_informative=10,
        n_redundant=10,
        n_repeated=5,
        n_classes=2,
        random_state=42
    )

    X_df = pd.DataFrame(X_data, columns=[f'feature_{i}' for i in range(50)])

    # Apply feature selection
    selector = FeatureSelector(X_df, y_data, task='classification')

    # Apply different selection methods
    variance_features = selector.variance_threshold()
    print(f"Variance Threshold: {len(variance_features)} features selected")

```

```python
    univariate_features = selector.univariate_selection(k=15)
    print(f"Univariate Selection: {len(univariate_features)} features selected")

    rfe_features = selector.recursive_feature_elimination(n_features_to_select=10)
    print(f"RFE: {len(rfe_features)} features selected")

    lasso_features = selector.lasso_selection()
    print(f"Lasso: {len(lasso_features)} features selected")

    tree_features = selector.tree_based_selection()
    print(f"Tree-based: {len(tree_features)} features selected")

    correlation_features = selector.correlation_selection()
    print(f"Correlation: {len(correlation_features)} features selected")

    # Compare methods
    print("\nFeature Selection Methods Comparison:")
    print(selector.compare_methods())

    # Visualize results
    selector.visualize_selection()

    # Find common features across methods
    all_selected = []
    for method, results in selector.selection_results.items():
        all_selected.extend(results['selected_features'])

    feature_counts = Counter(all_selected)

    # Features selected by multiple methods
    common_features = pd.DataFrame({
        'Feature': list(feature_counts.keys()),
        'Count': list(feature_counts.values())
    }).sort_values('Count', ascending=False)

    print(f"\nFeatures selected by multiple methods:")
    print(common_features[common_features['Count'] >= 3].head(10))
```

Listing 3.8: Comprehensive Feature Selection

3.6 Chapter Summary

In this chapter, we covered comprehensive data preprocessing and feature engineering techniques:

- **Data Quality and Cleaning:**
  - Identifying and handling missing data
  - Detecting and treating outliers
  - Dealing with duplicates and inconsistencies
- **Feature Scaling:**
  - Standardization (Z-score normalization)
  - Min-Max scaling

  - Robust scaling
  - Normalization techniques

- **Feature Transformation:**
  - Log and power transformations
  - Box-Cox and Yeo-Johnson transformations
  - Discretization and binning

- **Feature Engineering:**
  - Creating polynomial and interaction features
  - Statistical aggregations
  - Domain-specific features
  - Temporal features

- **Feature Selection:**
  - Filter methods (variance, correlation, statistical tests)
  - Wrapper methods (RFE)
  - Embedded methods (Lasso, tree-based)

3.7 Exercises

3.7.1 Conceptual Questions

1. Explain the difference between MCAR, MAR, and MNAR missing data patterns.
2. When should you use robust scaling instead of standard scaling?
3. Why might Box-Cox transformation fail, and when would you use Yeo-Johnson instead?
4. Describe the trade-off between creating many features and the curse of dimensionality.

3.7.2 Programming Exercises

1. **Missing Data Imputation Comparison:**

```python
def compare_imputation_methods(df_with_missing, true_values):
    """
    Compare different imputation methods against known true values
    Returns: Dictionary with method names and their RMSE
    """
    # Your implementation here
    pass

```

Listing 3.9: Imputation Comparison Function

2. **Automated Feature Engineering:**

```python
def auto_feature_engineer(df, target_col):
    """
    Automatically create features based on data types
    Returns: DataFrame with original and engineered features
    """
    # Your implementation here
    pass

```

Listing 3.10: Automated Feature Engineering

3. **Custom Transformer:**

```python
from sklearn.base import BaseEstimator, TransformerMixin

class CustomScaler(BaseEstimator, TransformerMixin):
    """
    Create a custom scaler that's robust to outliers
    """
    def fit(self, X, y=None):
        # Your implementation here
        return self

    def transform(self, X):
        # Your implementation here
        pass

```

Listing 3.11: Custom Scaler Implementation

3.7.3 Lab Assignment

Project: Build a Complete Data Preprocessing Pipeline

Using the Adult Income dataset, create a comprehensive preprocessing pipeline:

```python
# Load the Adult Income dataset
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"
columns = ['age', 'workclass', 'fnlwgt', 'education', 'education-num',
           'marital-status', 'occupation', 'relationship', 'race', 'sex',
           'capital-gain', 'capital-loss', 'hours-per-week', 'native-country', 'income']

adult_df = pd.read_csv(url, names=columns, na_values=' ?', skipinitialspace=True)
```

Listing 3.12: Adult Income Dataset Preprocessing

Tasks:

1. Perform comprehensive EDA and identify data quality issues
2. Handle missing values appropriately
3. Encode categorical variables
4. Create at least 10 meaningful engineered features

5. Apply appropriate scaling techniques
6. Use at least 3 feature selection methods
7. Create a final preprocessed dataset ready for modeling
8. Document your decisions and rationale

Chapter 4

Classification Algorithms

4.1 Learning Objectives

Learning Objectives

By the end of this chapter, you will:

- Master fundamental classification algorithms
- Understand decision boundaries and how different algorithms create them
- Implement classifiers from scratch
- Handle multi-class and imbalanced classification problems
- Evaluate and compare classifier performance

4.2 Introduction to Classification

Classification is a supervised learning task where the goal is to predict discrete class labels. Unlike regression which predicts continuous values, classification deals with categorical outcomes.

Definition

Classification involves predicting categorical class labels based on input features. The model learns a decision boundary that separates different classes in the feature space.

4.2.1 Types of Classification Problems

- **Binary Classification:** Two classes (e.g., spam/ham, sick/healthy)
- **Multi-class Classification:** More than two classes (e.g., digit recognition 0-9)
- **Multi-label Classification:** Multiple labels per instance (e.g., movie genres)

4.2.2 Key Concepts

- **Decision Boundary:** The surface that separates different classes
- **Probability Estimation:** Some classifiers provide class probabilities

- **Linearity vs Non-linearity:** Different algorithms make different assumptions about the decision boundary

4.3 Logistic Regression

4.3.1 Theory and Mathematical Foundation

Logistic regression models the probability of a binary outcome using the logistic (sigmoid) function, despite its name, it's a classification algorithm.

Definition

Logistic Regression estimates the probability that an instance belongs to a particular class using the sigmoid function to map linear combinations to probabilities between 0 and 1.

Mathematical Formulation

Sigmoid Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Model:

$$P(y = 1|x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}$$

Loss Function (Binary Cross-Entropy):

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^{m} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Gradients:

$$\frac{\partial J}{\partial w} = \frac{1}{m} X^T (\hat{y} - y)$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)$$

4.3.2 Binary Logistic Regression Implementation

Example

[Logistic Regression from Scratch]

```python
import numpy as np
from scipy.special import expit  # Sigmoid function
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

class LogisticRegressionFromScratch:
    """
    Logistic Regression implementation from scratch
    """

    def __init__(self, learning_rate=0.01, n_iterations=1000,
    regularization=None, lambda_reg=0.01):
```

```python
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.regularization = regularization  # None, 'l2', or 'l1'
        self.lambda_reg = lambda_reg
        self.losses = []

    def sigmoid(self, z):
        """Sigmoid activation function"""
        return expit(z)  # More numerically stable than 1/(1+np.exp(-z))

    def cost_function(self, X, y, theta):
        """Calculate the cost (binary cross-entropy)"""
        m = len(y)
        z = X.dot(theta)
        predictions = self.sigmoid(z)

        # Avoid log(0)
        epsilon = 1e-7
        predictions = np.clip(predictions, epsilon, 1 - epsilon)

        # Binary cross-entropy
        cost = -(1/m) * np.sum(y * np.log(predictions) + (1 - y) * np.log
    (1 - predictions))

        # Add regularization
        if self.regularization == 'l2':
            cost += (self.lambda_reg / (2 * m)) * np.sum(theta[1:]**2)
        elif self.regularization == 'l1':
            cost += (self.lambda_reg / m) * np.sum(np.abs(theta[1:]))

        return cost

    def gradient(self, X, y, theta):
        """Calculate the gradient"""
        m = len(y)
        z = X.dot(theta)
        predictions = self.sigmoid(z)

        gradient = (1/m) * X.T.dot(predictions - y)

        # Add regularization to gradient (except for bias term)
        if self.regularization == 'l2':
            gradient[1:] += (self.lambda_reg / m) * theta[1:]
        elif self.regularization == 'l1':
            gradient[1:] += (self.lambda_reg / m) * np.sign(theta[1:])

        return gradient

    def fit(self, X, y):
        """Train the logistic regression model"""

        # Add intercept term
        X = np.column_stack([np.ones(len(X)), X])

        # Initialize parameters
        self.theta = np.zeros(X.shape[1])

        # Gradient descent
        for i in range(self.n_iterations):
            # Calculate cost
```

```python
            cost = self.cost_function(X, y, self.theta)
            self.losses.append(cost)

            # Calculate gradient
            grad = self.gradient(X, y, self.theta)

            # Update parameters
            self.theta -= self.learning_rate * grad

            # Print progress
            if i % 100 == 0:
                print(f"Iteration {i}, Cost: {cost:.4f}")

        return self

    def predict_proba(self, X):
        """Predict probabilities"""
        X = np.column_stack([np.ones(len(X)), X])
        return self.sigmoid(X.dot(self.theta))

    def predict(self, X, threshold=0.5):
        """Predict classes"""
        return (self.predict_proba(X) >= threshold).astype(int)

    def decision_boundary(self, X):
        """Get decision boundary (for 2D features)"""
        # For 2D: w0 + w1*x1 + w2*x2 = 0
        # x2 = -(w0 + w1*x1) / w2
        if X.shape[1] == 2:
            x1_min, x1_max = X[:, 0].min() - 1, X[:, 0].max() + 1
            x1_boundary = np.linspace(x1_min, x1_max, 100)
            x2_boundary = -(self.theta[0] + self.theta[1] * x1_boundary) / self.theta[2]
            return x1_boundary, x2_boundary
        else:
            raise ValueError("Decision boundary visualization requires 2D features")

# Demonstration
if __name__ == "__main__":
    # Create dataset
    X, y = make_classification(n_samples=1000, n_features=2, n_informative=2,
                               n_redundant=0, n_clusters_per_class=1,
                               random_state=42)

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

    # Train model
    print("Training Logistic Regression...")
    log_reg = LogisticRegressionFromScratch(learning_rate=0.1, n_iterations=1000,
                                            regularization='l2', lambda_reg=0.1)
    log_reg.fit(X_train, y_train)

    # Predictions
    y_pred = log_reg.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
```

```python
    print(f"\nAccuracy: {accuracy:.3f}")
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))

    # Visualization
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

    # Plot 1: Training data and decision boundary
    axes[0].scatter(X_train[y_train == 0][:, 0], X_train[y_train == 0][:, 1],
                    c='blue', label='Class 0', alpha=0.5)
    axes[0].scatter(X_train[y_train == 1][:, 0], X_train[y_train == 1][:, 1],
                    c='red', label='Class 1', alpha=0.5)

    # Decision boundary
    x1_boundary, x2_boundary = log_reg.decision_boundary(X_train)
    axes[0].plot(x1_boundary, x2_boundary, 'g--', label='Decision Boundary')
    axes[0].set_xlabel('Feature 1')
    axes[0].set_ylabel('Feature 2')
    axes[0].set_title('Training Data with Decision Boundary')
    axes[0].legend()

    # Plot 2: Loss curve
    axes[1].plot(log_reg.losses)
    axes[1].set_xlabel('Iteration')
    axes[1].set_ylabel('Loss')
    axes[1].set_title('Training Loss Curve')
    axes[1].grid(True, alpha=0.3)

    # Plot 3: Probability contours
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                         np.linspace(y_min, y_max, 100))

    Z = log_reg.predict_proba(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    contour = axes[2].contourf(xx, yy, Z, levels=20, cmap='RdBu', alpha=0.8)
    axes[2].scatter(X_test[y_test == 0][:, 0], X_test[y_test == 0][:, 1],
                    c='blue', edgecolor='black', label='Class 0')
    axes[2].scatter(X_test[y_test == 1][:, 0], X_test[y_test == 1][:, 1],
                    c='red', edgecolor='black', label='Class 1')
    axes[2].set_xlabel('Feature 1')
    axes[2].set_ylabel('Feature 2')
    axes[2].set_title('Probability Contours (Test Data)')
    axes[2].legend()
    plt.colorbar(contour, ax=axes[2])

    plt.tight_layout()
    plt.show()
```

Listing 4.1: Binary Logistic Regression Implementation

4.3.3 Multi-class Logistic Regression

For multi-class problems, we extend logistic regression using strategies like One-vs-Rest (OvR) or softmax regression.

Example

[Multi-class Logistic Regression]

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import StandardScaler
from logistic_regression_binary import LogisticRegressionFromScratch

class MultiClassLogisticRegression:
    """
    Multi-class Logistic Regression using One-vs-Rest strategy
    """

    def __init__(self, learning_rate=0.01, n_iterations=1000):
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.classifiers = {}
        self.classes = None

    def fit(self, X, y):
        """Train one binary classifier for each class"""
        self.classes = np.unique(y)

        for cls in self.classes:
            print(f"Training classifier for class {cls}")
            # Create binary labels
            binary_y = (y == cls).astype(int)

            # Train binary classifier
            classifier = LogisticRegressionFromScratch(
                learning_rate=self.learning_rate,
                n_iterations=self.n_iterations
            )
            classifier.fit(X, binary_y)
            self.classifiers[cls] = classifier

        return self

    def predict_proba(self, X):
        """Predict probabilities for each class"""
        probas = np.zeros((len(X), len(self.classes)))

        for i, cls in enumerate(self.classes):
            probas[:, i] = self.classifiers[cls].predict_proba(X)

        # Normalize to sum to 1
        probas = probas / probas.sum(axis=1, keepdims=True)

        return probas

    def predict(self, X):
        """Predict class labels"""
```

```python
        probas = self.predict_proba(X)
        return self.classes[np.argmax(probas, axis=1)]

# Demonstration with iris dataset
if __name__ == "__main__":
    iris = load_iris()
    X_iris = iris.data[:, [2, 3]]  # Use only petal length and width for visualization
    y_iris = iris.target

    # Standardize features
    scaler = StandardScaler()
    X_iris_scaled = scaler.fit_transform(X_iris)

    # Split data
    X_train, X_test, y_train, y_test = train_test_split(
        X_iris_scaled, y_iris, test_size=0.3, random_state=42
    )

    # Train multi-class model
    print("Training Multi-class Logistic Regression...")
    multi_log_reg = MultiClassLogisticRegression(learning_rate=0.1, n_iterations=500)
    multi_log_reg.fit(X_train, y_train)

    # Predictions
    y_pred = multi_log_reg.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    print(f"\nMulti-class Accuracy: {accuracy:.3f}")
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred, target_names=iris.target_names))

    # Visualization
    plt.figure(figsize=(12, 5))

    # Plot decision boundaries
    x_min, x_max = X_iris_scaled[:, 0].min() - 1, X_iris_scaled[:, 0].max() + 1
    y_min, y_max = X_iris_scaled[:, 1].min() - 1, X_iris_scaled[:, 1].max() + 1
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                         np.linspace(y_min, y_max, 100))

    Z = multi_log_reg.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3, cmap='Set1')

    # Plot training points
    for i, color, marker in zip([0, 1, 2], ['red', 'blue', 'green'], ['o', 's', '^']):
        plt.scatter(X_train[y_train == i, 0], X_train[y_train == i, 1],
                    c=color, marker=marker, label=iris.target_names[i],
                    edgecolor='black', s=50)

    plt.xlabel('Petal Length (standardized)')
    plt.ylabel('Petal Width (standardized)')
    plt.title('Multi-class Logistic Regression Decision Boundaries')
    plt.legend()
```

```python
    plt.show()

    # Show probability distributions for a few test samples
    print("\nProbability predictions for first 5 test samples:")
    test_probas = multi_log_reg.predict_proba(X_test[:5])
    for i, (true_class, pred_probas) in enumerate(zip(y_test[:5], test_probas)):
        print(f"Sample {i+1}: True class: {iris.target_names[true_class]}")
        for j, prob in enumerate(pred_probas):
            print(f"  {iris.target_names[j]}: {prob:.3f}")
        print()
```

Listing 4.2: Multi-class Logistic Regression

4.4 Decision Trees

4.4.1 Theory and Concepts

Decision trees recursively partition the feature space using if-then rules, creating a flowchart-like structure.

Definition

A **Decision Tree** is a tree-like model where each internal node represents a feature test, each branch represents the outcome, and each leaf node holds a class prediction.

Splitting Criteria

Gini Impurity:

$$Gini = 1 - \sum_{i=1}^{c} p_i^2$$

Entropy:

$$Entropy = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

Information Gain:

$$IG = Entropy(parent) - \sum_{j} \frac{n_j}{n} Entropy(child_j)$$

4.4.2 Decision Tree Implementation

Example

[Decision Tree from Scratch]

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification, make_moons
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from collections import Counter
```

```python

class Node:
    """A node in the decision tree"""

    def __init__(self, feature=None, threshold=None, left=None, right=None,
                 value=None, samples=None, impurity=None):
        self.feature = feature          # Feature index for splitting
        self.threshold = threshold      # Threshold value for splitting
        self.left = left               # Left child node
        self.right = right             # Right child node
        self.value = value             # Class prediction for leaf nodes
        self.samples = samples         # Number of samples in node
        self.impurity = impurity       # Impurity measure of node

    def is_leaf(self):
        """Check if node is a leaf"""
        return self.value is not None

class DecisionTreeFromScratch:
    """
    Decision Tree Classifier implementation from scratch
    """

    def __init__(self, max_depth=10, min_samples_split=2, min_samples_leaf=1,
                 criterion='gini', max_features=None):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.min_samples_leaf = min_samples_leaf
        self.criterion = criterion
        self.max_features = max_features
        self.root = None
        self.n_features_ = None
        self.n_classes_ = None
        self.feature_importances_ = None

    def gini_impurity(self, y):
        """Calculate Gini impurity"""
        if len(y) == 0:
            return 0
        proportions = np.bincount(y) / len(y)
        return 1 - np.sum(proportions ** 2)

    def entropy(self, y):
        """Calculate entropy"""
        if len(y) == 0:
            return 0
        proportions = np.bincount(y) / len(y)
        # Avoid log(0)
        proportions = proportions[proportions > 0]
        return -np.sum(proportions * np.log2(proportions))

    def calculate_impurity(self, y):
        """Calculate impurity based on chosen criterion"""
        if self.criterion == 'gini':
            return self.gini_impurity(y)
        elif self.criterion == 'entropy':
            return self.entropy(y)
        else:
```

```python
            raise ValueError(f"Unknown criterion: {self.criterion}")

    def information_gain(self, parent, left_child, right_child):
        """Calculate information gain or Gini reduction"""
        n = len(parent)
        n_left = len(left_child)
        n_right = len(right_child)

        if n_left == 0 or n_right == 0:
            return 0

        parent_impurity = self.calculate_impurity(parent)
        left_impurity = self.calculate_impurity(left_child)
        right_impurity = self.calculate_impurity(right_child)

        weighted_impurity = (n_left / n) * left_impurity + (n_right / n) * right_impurity

        return parent_impurity - weighted_impurity

    def find_best_split(self, X, y):
        """Find the best split for a node"""
        best_gain = -1
        best_feature = None
        best_threshold = None

        n_features = X.shape[1]

        for feature in range(n_features):
            # Get unique values for this feature
            thresholds = np.unique(X[:, feature])

            # Try all possible thresholds
            for threshold in thresholds:
                # Split data
                left_mask = X[:, feature] <= threshold
                right_mask = ~left_mask

                # Check minimum samples leaf constraint
                if (np.sum(left_mask) < self.min_samples_leaf or
                    np.sum(right_mask) < self.min_samples_leaf):
                    continue

                # Calculate information gain
                gain = self.information_gain(
                    y,
                    y[left_mask],
                    y[right_mask]
                )

                if gain > best_gain:
                    best_gain = gain
                    best_feature = feature
                    best_threshold = threshold

        return best_feature, best_threshold, best_gain

    def build_tree(self, X, y, depth=0):
        """Recursively build the decision tree"""

```

```
        n_samples = len(y)
        n_classes = len(np.unique(y))

        # Calculate node impurity
        node_impurity = self.calculate_impurity(y)

        # Get majority class for this node
        class_counts = np.bincount(y)
        majority_class = np.argmax(class_counts)

        # Stopping criteria
        if (depth >= self.max_depth or
            n_classes == 1 or
            n_samples < self.min_samples_split or
            node_impurity == 0):
            # Return leaf node
            return Node(value=majority_class, samples=n_samples, impurity=node_impurity)

        # Find best split
        best_feature, best_threshold, best_gain = self.find_best_split(X, y)

        # If no valid split found, return leaf node
        if best_gain <= 0:
            return Node(value=majority_class, samples=n_samples, impurity=node_impurity)

        # Split data
        left_mask = X[:, best_feature] <= best_threshold
        right_mask = ~left_mask

        # Recursively build left and right subtrees
        left_child = self.build_tree(X[left_mask], y[left_mask], depth + 1)
        right_child = self.build_tree(X[right_mask], y[right_mask], depth + 1)

        return Node(
            feature=best_feature,
            threshold=best_threshold,
            left=left_child,
            right=right_child,
            samples=n_samples,
            impurity=node_impurity
        )

    def fit(self, X, y):
        """Train the decision tree"""
        self.n_features_ = X.shape[1]
        self.n_classes_ = len(np.unique(y))
        self.feature_importances_ = np.zeros(self.n_features_)

        self.root = self.build_tree(X, y)
        return self

    def predict_sample(self, x, node):
        """Predict class for a single sample"""
        if node.is_leaf():
            return node.value
```

```python

        if x[node.feature] <= node.threshold:
            return self.predict_sample(x, node.left)
        else:
            return self.predict_sample(x, node.right)

    def predict(self, X):
        """Predict classes for multiple samples"""
        predictions = []
        for x in X:
            predictions.append(self.predict_sample(x, self.root))
        return np.array(predictions)

    def get_depth(self, node=None):
        """Get the depth of the tree"""
        if node is None:
            node = self.root

        if node.is_leaf():
            return 0

        return 1 + max(self.get_depth(node.left), self.get_depth(node.right))

# Demonstration
if __name__ == "__main__":
    # Create different datasets for demonstration
    np.random.seed(42)

    # Dataset 1: Two moons (non-linear)
    X_moons, y_moons = make_moons(n_samples=500, noise=0.3, random_state=42)

    # Dataset 2: Linear separable
    X_linear, y_linear = make_classification(n_samples=500, n_features=2,
                                             n_informative=2, n_redundant=0,
                                             n_clusters_per_class=1, random_state=42)

    datasets = [
        ("Two Moons", X_moons, y_moons),
        ("Linear Separable", X_linear, y_linear)
    ]

    fig, axes = plt.subplots(2, 3, figsize=(15, 10))

    for idx, (name, X, y) in enumerate(datasets):
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.3, random_state=42
        )

        # Train decision tree with different depths
        depths = [3, 5, None]

        for depth_idx, max_depth in enumerate(depths):
            dt = DecisionTreeFromScratch(
                max_depth=max_depth,
                min_samples_split=10,
                criterion='gini'
```

```
            )
            dt.fit(X_train, y_train)

            # Predictions
            y_pred = dt.predict(X_test)
            accuracy = accuracy_score(y_test, y_pred)

            # Plot decision boundaries
            ax = axes[idx, depth_idx]

            x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
            y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5

            xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                                 np.linspace(y_min, y_max, 100))

            Z = dt.predict(np.c_[xx.ravel(), yy.ravel()])
            Z = Z.reshape(xx.shape)

            ax.contourf(xx, yy, Z, alpha=0.4, cmap='RdBu')
            ax.scatter(X_test[y_test == 0][:, 0], X_test[y_test == 0][:, 1],
                       c='blue', label='Class 0', edgecolor='black', s=50)
            ax.scatter(X_test[y_test == 1][:, 0], X_test[y_test == 1][:, 1],
                       c='red', label='Class 1', edgecolor='black', s=50)

            depth_str = f"Depth={max_depth}" if max_depth else "No limit"
            ax.set_title(f'{name}\n{depth_str}, Acc={accuracy:.3f}')
            ax.set_xlabel('Feature 1')
            ax.set_ylabel('Feature 2')
            ax.legend()

    plt.tight_layout()
    plt.show()

    # Demonstrate overfitting with different max_depth values
    print("\nOverfitting Analysis:")
    print("-" * 50)

    X, y = make_classification(n_samples=500, n_features=2, random_state=42)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

    depths = range(1, 21)
    train_scores = []
    test_scores = []

    for depth in depths:
        dt = DecisionTreeFromScratch(max_depth=depth, criterion='entropy')
        dt.fit(X_train, y_train)

        train_score = accuracy_score(y_train, dt.predict(X_train))
        test_score = accuracy_score(y_test, dt.predict(X_test))

        train_scores.append(train_score)
        test_scores.append(test_score)

    plt.figure(figsize=(10, 6))
```

```
    plt.plot(depths, train_scores, 'o-', label='Training Accuracy',
    linewidth=2)
    plt.plot(depths, test_scores, 's-', label='Testing Accuracy',
    linewidth=2)
    plt.xlabel('Max Depth')
    plt.ylabel('Accuracy')
    plt.title('Decision Tree: Overfitting Analysis')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()
```

Listing 4.3: Decision Tree Implementation

4.4.3 Pruning and Regularization

Decision trees are prone to overfitting. Pruning techniques help prevent this:

- **Pre-pruning:** Stop growing early (max depth, min samples)
- **Post-pruning:** Remove branches after full growth

Example

[Decision Tree Pruning]

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from decision_trees import DecisionTreeFromScratch, Node

class PrunedDecisionTree(DecisionTreeFromScratch):
    """
    Decision Tree with post-pruning capabilities
    """

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.pruning_alpha = 0.0

    def calculate_leaf_errors(self, node, X, y):
        """Calculate the number of misclassifications if node becomes a
    leaf"""
        predictions = np.full(len(y), node.value)
        return np.sum(predictions != y)

    def calculate_subtree_errors(self, node, X, y):
        """Calculate total errors in subtree"""
        if node.is_leaf():
            predictions = np.full(len(y), node.value)
            return np.sum(predictions != y)

        # Split data
        left_mask = X[:, node.feature] <= node.threshold
        right_mask = ~left_mask

        left_errors = self.calculate_subtree_errors(
```

```
            node.left, X[left_mask], y[left_mask]
        ) if np.any(left_mask) else 0

        right_errors = self.calculate_subtree_errors(
            node.right, X[right_mask], y[right_mask]
        ) if np.any(right_mask) else 0

        return left_errors + right_errors

    def prune_tree(self, node, X, y, X_val, y_val):
        """Recursively prune the tree using validation data"""
        if node.is_leaf():
            return node

        # Recursively prune children first
        left_mask = X[:, node.feature] <= node.threshold
        right_mask = ~left_mask

        if np.any(left_mask):
            node.left = self.prune_tree(
                node.left, X[left_mask], y[left_mask],
                X_val, y_val
            )

        if np.any(right_mask):
            node.right = self.prune_tree(
                node.right, X[right_mask], y[right_mask],
                X_val, y_val
            )

        # Check if pruning this node improves validation accuracy
        val_mask_left = X_val[:, node.feature] <= node.threshold
        val_mask_right = ~val_mask_left

        # Errors if we keep the subtree
        subtree_errors = 0
        if np.any(val_mask_left):
            subtree_errors += self.calculate_subtree_errors(
                node.left, X_val[val_mask_left], y_val[val_mask_left]
            )
        if np.any(val_mask_right):
            subtree_errors += self.calculate_subtree_errors(
                node.right, X_val[val_mask_right], y_val[val_mask_right]
            )

        # Errors if we prune (make this node a leaf)
        leaf_errors = self.calculate_leaf_errors(node, X_val, y_val)

        # Prune if it doesn't increase errors
        if leaf_errors <= subtree_errors:
            # Convert to leaf node
            return Node(value=node.value, samples=node.samples, impurity=
    node.impurity)

        return node

    def fit_and_prune(self, X, y, X_val, y_val):
        """Fit the tree and then prune using validation data"""
        # First fit the tree normally
        self.fit(X, y)
```

```python

        # Then prune using validation data
        self.root = self.prune_tree(self.root, X, y, X_val, y_val)

        return self

# Demonstrate pruning effect
if __name__ == "__main__":
    X, y = make_classification(n_samples=1000, n_features=20, n_informative=10,
                               n_redundant=5, n_clusters_per_class=2, random_state=42)

    X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.4, random_state=42)
    X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)

    # Train unpruned tree
    print("Training unpruned tree...")
    unpruned_tree = DecisionTreeFromScratch(max_depth=None, min_samples_split=2)
    unpruned_tree.fit(X_train, y_train)

    # Train pruned tree
    print("Training and pruning tree...")
    pruned_tree = PrunedDecisionTree(max_depth=None, min_samples_split=2)
    pruned_tree.fit_and_prune(X_train, y_train, X_val, y_val)

    # Compare performance
    models = {
        'Unpruned': unpruned_tree,
        'Pruned': pruned_tree
    }

    print("\nPruning Comparison:")
    print("-" * 50)

    for name, model in models.items():
        train_acc = accuracy_score(y_train, model.predict(X_train))
        val_acc = accuracy_score(y_val, model.predict(X_val))
        test_acc = accuracy_score(y_test, model.predict(X_test))

        print(f"{name} Tree:")
        print(f"  Train Accuracy: {train_acc:.3f}")
        print(f"  Val Accuracy:   {val_acc:.3f}")
        print(f"  Test Accuracy:  {test_acc:.3f}")
        print(f"  Tree Depth:     {model.get_depth()}")
        print()

    # Visualize the effect of different pruning strengths
    print("\nPruning Strength Analysis:")
    print("-" * 50)

    # Create a simpler dataset for visualization
    X_simple, y_simple = make_classification(n_samples=300, n_features=2,
                                             n_informative=2, n_redundant=0,
                                             random_state=42)
    X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
```

```python
        X_simple, y_simple, test_size=0.3, random_state=42
    )
    X_train_s, X_val_s, y_train_s, y_val_s = train_test_split(
        X_train_s, y_train_s, test_size=0.2, random_state=42
    )

    max_depths = [2, 5, 10, None]

    fig, axes = plt.subplots(2, 4, figsize=(16, 8))

    for i, max_depth in enumerate(max_depths):
        # Unpruned
        dt_unpruned = DecisionTreeFromScratch(max_depth=max_depth)
        dt_unpruned.fit(X_train_s, y_train_s)

        # Pruned
        dt_pruned = PrunedDecisionTree(max_depth=max_depth)
        dt_pruned.fit_and_prune(X_train_s, y_train_s, X_val_s, y_val_s)

        trees = [dt_unpruned, dt_pruned]
        titles = [f'Unpruned (Depth={max_depth})', f'Pruned (Depth={max_depth})']

        for j, (tree, title) in enumerate(zip(trees, titles)):
            ax = axes[j, i]

            # Plot decision boundary
            x_min, x_max = X_simple[:, 0].min() - 1, X_simple[:, 0].max() + 1
            y_min, y_max = X_simple[:, 1].min() - 1, X_simple[:, 1].max() + 1
            xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                                 np.linspace(y_min, y_max, 100))

            Z = tree.predict(np.c_[xx.ravel(), yy.ravel()])
            Z = Z.reshape(xx.shape)

            ax.contourf(xx, yy, Z, alpha=0.3, cmap='RdBu')
            ax.scatter(X_test_s[y_test_s == 0][:, 0], X_test_s[y_test_s == 0][:, 1],
                       c='blue', label='Class 0', alpha=0.6)
            ax.scatter(X_test_s[y_test_s == 1][:, 0], X_test_s[y_test_s == 1][:, 1],
                       c='red', label='Class 1', alpha=0.6)

            test_acc = accuracy_score(y_test_s, tree.predict(X_test_s))
            ax.set_title(f'{title}\nTest Acc: {test_acc:.3f}')
            ax.set_xlabel('Feature 1')
            ax.set_ylabel('Feature 2')

    plt.tight_layout()
    plt.show()
```

Listing 4.4: Decision Tree Pruning Techniques

4.5 Support Vector Machines (SVM)

4.5.1 Theory and Mathematical Foundation

SVMs find the optimal hyperplane that maximizes the margin between classes.

Definition

Support Vector Machines are maximum-margin classifiers that find the hyperplane with the largest distance to the nearest training data points of any class.

Optimization Problem

Linear SVM:

$$\min_{w,b} \frac{1}{2}||w||^2$$

$$\text{subject to } y_i(w^T x_i + b) \geq 1, \quad \forall i$$

Soft Margin SVM:

$$\min_{w,b,\xi} \frac{1}{2}||w||^2 + C\sum_{i=1}^{n} \xi_i$$

$$\text{subject to } y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Kernel Trick

The kernel trick allows SVMs to handle non-linear relationships:

- **Linear:** $K(x_i, x_j) = x_i^T x_j$
- **Polynomial:** $K(x_i, x_j) = (x_i^T x_j + r)^d$
- **RBF:** $K(x_i, x_j) = \exp(-||x_i - x_j||^2)$
- **Sigmoid:** $K(x_i, x_j) = \tanh(x_i^T x_j + r)$

4.5.2 SVM Implementation

Example

[SVM Implementation and Visualization]

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_classification, make_circles, make_moons
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import pandas as pd

class SVMDemonstration:
    """
    Comprehensive SVM demonstration class
    """

    def __init__(self):
        self.models = {}
        self.datasets = {}

```

```python
    def create_datasets(self):
        """Create different types of datasets for SVM demonstration"""

        # Linear separable data
        X_linear, y_linear = make_classification(
            n_samples=100, n_features=2, n_informative=2,
            n_redundant=0, n_clusters_per_class=1,
            class_sep=2.0, random_state=42
        )

        # Non-linear: circles
        X_circles, y_circles = make_circles(
            n_samples=100, noise=0.1, factor=0.5, random_state=42
        )

        # Non-linear: moons
        X_moons, y_moons = make_moons(
            n_samples=100, noise=0.15, random_state=42
        )

        self.datasets = {
            'Linear': (X_linear, y_linear),
            'Circles': (X_circles, y_circles),
            'Moons': (X_moons, y_moons)
        }

        return self.datasets

    def plot_decision_boundary(self, X, y, model, title, ax):
        """Plot SVM decision boundary and support vectors"""

        h = 0.02  # step size in mesh
        x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
        y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
        xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                             np.arange(y_min, y_max, h))

        Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
        Z = Z.reshape(xx.shape)

        # Plot decision boundary
        ax.contourf(xx, yy, Z, alpha=0.4, cmap='RdBu')

        # Plot data points
        ax.scatter(X[:, 0], X[:, 1], c=y, cmap='RdBu',
                            edgecolor='black', s=50)

        # Highlight support vectors
        if hasattr(model, 'support_vectors_'):
            ax.scatter(model.support_vectors_[:, 0],
                       model.support_vectors_[:, 1],
                       s=100, facecolors='none', edgecolors='green',
                       linewidth=2, label='Support Vectors')

        ax.set_xlabel('Feature 1')
        ax.set_ylabel('Feature 2')
        ax.set_title(title)
        ax.legend()

    def demonstrate_linear_svm(self):
```

```
        """Demonstrate linear SVM on separable data"""

        X, y = self.datasets['Linear']

        # Standardize features
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

        # Different C values
        C_values = [0.01, 0.1, 1, 10, 100]

        fig, axes = plt.subplots(2, 3, figsize=(15, 10))
        axes = axes.ravel()

        for i, C in enumerate(C_values):
            if i < len(axes):
                # Create and train SVM
                svm = SVC(kernel='linear', C=C)
                svm.fit(X_scaled, y)

                # Calculate accuracy
                accuracy = svm.score(X_scaled, y)

                # Plot
                title = f'Linear SVM (C={C})\nAccuracy: {accuracy:.3f}, SV
    : {len(svm.support_vectors_)}'
                self.plot_decision_boundary(X_scaled, y, svm, title, axes[
    i])

        plt.suptitle('Linear SVM with Different Regularization Parameters'
    , fontsize=16)
        plt.tight_layout()
        plt.show()

    def demonstrate_kernels(self):
        """Demonstrate different kernel functions"""

        # Use circles dataset
        X, y = self.datasets['Circles']

        # Standardize
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

        # Kernels to test
        kernels = [
            ('linear', 'Linear'),
            ('poly', 'Polynomial (d=3)'),
            ('rbf', 'RBF'),
            ('sigmoid', 'Sigmoid')
        ]

        fig, axes = plt.subplots(2, 2, figsize=(12, 10))
        axes = axes.ravel()

        for i, (kernel, kernel_name) in enumerate(kernels):
            if i < len(axes):
                # Create and train SVM
                if kernel == 'poly':
                    svm = SVC(kernel=kernel, degree=3, C=1.0, gamma='scale
```

```
    ')
                else:
                    svm = SVC(kernel=kernel, C=1.0, gamma='scale')

                svm.fit(X_scaled, y)

                # Calculate accuracy
                accuracy = svm.score(X_scaled, y)

                # Plot
                title = f'{kernel_name} Kernel\nAccuracy: {accuracy:.3f}'
                self.plot_decision_boundary(X_scaled, y, svm, title, axes[
    i])

        plt.suptitle('SVM with Different Kernel Functions', fontsize=16)
        plt.tight_layout()
        plt.show()

# Demonstration
if __name__ == "__main__":
    svm_demo = SVMDemonstration()
    svm_demo.create_datasets()

    print("Demonstrating Linear SVM...")
    svm_demo.demonstrate_linear_svm()

    print("Demonstrating different kernels...")
    svm_demo.demonstrate_kernels()
```

Listing 4.5: SVM Implementation

4.5.3 Kernel Methods and Hyperparameter Tuning

Example

[SVM Kernels and Parameter Tuning]

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import accuracy_score, classification_report
import pandas as pd

class SVMHyperparameterTuning:
    """
    SVM hyperparameter tuning and kernel comparison
    """

    def __init__(self):
        self.results = {}

    def create_complex_dataset(self):
        """Create a complex dataset for kernel comparison"""
        # Create a dataset that requires non-linear separation
```

```
        X, y = make_classification(
            n_samples=500, n_features=2, n_informative=2, n_redundant=0,
            n_clusters_per_class=2, class_sep=1.5, random_state=42
        )

        # Add some non-linearity
        X = X + 0.3 * np.random.randn(500, 2)
        return X, y

    def tune_linear_svm(self, X, y):
        """Tune hyperparameters for linear SVM"""
        param_grid = {
            'C': [0.001, 0.01, 0.1, 1, 10, 100, 1000]
        }

        grid_search = GridSearchCV(
            SVC(kernel='linear'),
            param_grid,
            cv=5,
            scoring='accuracy',
            n_jobs=-1
        )
        grid_search.fit(X, y)

        return grid_search

    def tune_rbf_svm(self, X, y):
        """Tune hyperparameters for RBF kernel SVM"""
        param_grid = {
            'C': [0.001, 0.01, 0.1, 1, 10, 100, 1000],
            'gamma': [0.001, 0.01, 0.1, 1, 10, 100]
        }

        grid_search = GridSearchCV(
            SVC(kernel='rbf'),
            param_grid,
            cv=5,
            scoring='accuracy',
            n_jobs=-1
        )
        grid_search.fit(X, y)

        return grid_search

    def tune_poly_svm(self, X, y):
        """Tune hyperparameters for polynomial kernel SVM"""
        param_grid = {
            'C': [0.1, 1, 10, 100],
            'degree': [2, 3, 4, 5],
            'gamma': [0.1, 1, 10]
        }

        grid_search = GridSearchCV(
            SVC(kernel='poly'),
            param_grid,
            cv=5,
            scoring='accuracy',
            n_jobs=-1
        )
        grid_search.fit(X, y)
```

```python

        return grid_search

    def compare_kernels(self):
        """Comprehensive kernel comparison"""
        X, y = self.create_complex_dataset()

        # Split data
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.3, random_state=42
        )

        # Standardize
        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        # Tune each kernel
        print("Tuning Linear SVM...")
        linear_results = self.tune_linear_svm(X_train_scaled, y_train)

        print("Tuning RBF SVM...")
        rbf_results = self.tune_rbf_svm(X_train_scaled, y_train)

        print("Tuning Polynomial SVM...")
        poly_results = self.tune_poly_svm(X_train_scaled, y_train)

        # Store results
        self.results = {
            'Linear': {
                'best_params': linear_results.best_params_,
                'best_score': linear_results.best_score_,
                'model': linear_results.best_estimator_
            },
            'RBF': {
                'best_params': rbf_results.best_params_,
                'best_score': rbf_results.best_score_,
                'model': rbf_results.best_estimator_
            },
            'Polynomial': {
                'best_params': poly_results.best_estimator_.get_params(),
                'best_score': poly_results.best_score_,
                'model': poly_results.best_estimator_
            }
        }

        # Evaluate on test set
        print("\nKernel Comparison Results:")
        print("=" * 60)

        test_results = []
        for kernel_name, result in self.results.items():
            model = result['model']
            y_pred = model.predict(X_test_scaled)
            test_accuracy = accuracy_score(y_test, y_pred)

            test_results.append({
                'Kernel': kernel_name,
                'CV_Score': result['best_score'],
                'Test_Score': test_accuracy,
```

```
                'Parameters': str(result['best_params']),
                'Support_Vectors': len(model.support_vectors_)
            })

            print(f"\n{kernel_name} Kernel:")
            print(f"  Best CV Accuracy: {result['best_score']:.3f}")
            print(f"  Test Accuracy: {test_accuracy:.3f}")
            print(f"  Best Parameters: {result['best_params']}")
            print(f"  Support Vectors: {len(model.support_vectors_)}")

        return pd.DataFrame(test_results)

    def visualize_kernel_comparison(self):
        """Visualize the kernel comparison results"""
        X, y = self.create_complex_dataset()

        # Standardize
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

        fig, axes = plt.subplots(2, 3, figsize=(15, 10))

        # Plot original data
        axes[0, 0].scatter(X_scaled[y == 0, 0], X_scaled[y == 0, 1],
                           c='blue', label='Class 0', alpha=0.6)
        axes[0, 0].scatter(X_scaled[y == 1, 0], X_scaled[y == 1, 1],
                           c='red', label='Class 1', alpha=0.6)
        axes[0, 0].set_title('Original Data')
        axes[0, 0].set_xlabel('Feature 1')
        axes[0, 0].set_ylabel('Feature 2')
        axes[0, 0].legend()

        # Plot decision boundaries for each kernel
        kernels = ['Linear', 'RBF', 'Polynomial']

        for i, kernel in enumerate(kernels):
            model = self.results[kernel]['model']

            # Create mesh grid
            x_min, x_max = X_scaled[:, 0].min() - 1, X_scaled[:, 0].max() + 1
            y_min, y_max = X_scaled[:, 1].min() - 1, X_scaled[:, 1].max() + 1
            xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                                 np.linspace(y_min, y_max, 100))

            Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
            Z = Z.reshape(xx.shape)

            # Plot decision boundary
            contour = axes[0, i+1].contourf(xx, yy, Z, alpha=0.3, cmap='RdBu')
            axes[0, i+1].scatter(X_scaled[y == 0, 0], X_scaled[y == 0, 1],
                                 c='blue', alpha=0.6, s=30)
            axes[0, i+1].scatter(X_scaled[y == 1, 0], X_scaled[y == 1, 1],
                                 c='red', alpha=0.6, s=30)
            axes[0, i+1].scatter(model.support_vectors_[:, 0],
                                 model.support_vectors_[:, 1],
                                 s=80, facecolors='none', edgecolors='green')
```

```python
            axes[0, i+1].set_title(f'{kernel} Kernel\nTest Acc: {self.results[kernel]["best_score"]:.3f}')
            axes[0, i+1].set_xlabel('Feature 1')
            axes[0, i+1].set_ylabel('Feature 2')

        # Plot performance comparison
        kernel_names = list(self.results.keys())
        cv_scores = [self.results[k]['best_score'] for k in kernel_names]

        axes[1, 0].bar(kernel_names, cv_scores, color=['skyblue', 'lightcoral', 'lightgreen'])
        axes[1, 0].set_ylabel('Cross-Validation Accuracy')
        axes[1, 0].set_title('Kernel Performance Comparison')
        axes[1, 0].grid(True, alpha=0.3)

        # Add values on bars
        for i, v in enumerate(cv_scores):
            axes[1, 0].text(i, v + 0.01, f'{v:.3f}', ha='center', va='bottom')

        # Plot support vectors comparison
        support_vectors = [len(self.results[k]['model'].support_vectors_) for k in kernel_names]
        axes[1, 1].bar(kernel_names, support_vectors, color=['skyblue', 'lightcoral', 'lightgreen'])
        axes[1, 1].set_ylabel('Number of Support Vectors')
        axes[1, 1].set_title('Support Vectors Comparison')
        axes[1, 1].grid(True, alpha=0.3)

        # Add values on bars
        for i, v in enumerate(support_vectors):
            axes[1, 1].text(i, v + 5, str(v), ha='center', va='bottom')

        # Hide unused subplot
        axes[1, 2].axis('off')

        plt.tight_layout()
        plt.show()

# Demonstration
if __name__ == "__main__":
    svm_tuner = SVMHyperparameterTuning()

    print("Comparing SVM kernels with hyperparameter tuning...")
    results_df = svm_tuner.compare_kernels()

    print("\nDetailed Results DataFrame:")
    print("=" * 60)
    print(results_df)

    svm_tuner.visualize_kernel_comparison()
```

Listing 4.6: SVM Kernel Methods

4.6 Ensemble Methods

4.6.1 Introduction to Ensemble Learning

Ensemble methods combine multiple models to improve performance and robustness.

Definition

Ensemble Learning combines predictions from multiple models to produce better results than any single model could achieve.

4.6.2 Random Forests

Random forests build multiple decision trees and combine their predictions.

Example

[Random Forest Implementation]

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from decision_trees import DecisionTreeFromScratch
import pandas as pd

class RandomForestFromScratch:
    """
    Simple Random Forest implementation using our custom decision trees
    """

    def __init__(self, n_estimators=100, max_depth=10, max_features='sqrt'
    ,
                 min_samples_split=2, bootstrap=True, random_state=None):
        self.n_estimators = n_estimators
        self.max_depth = max_depth
        self.max_features = max_features
        self.min_samples_split = min_samples_split
        self.bootstrap = bootstrap
        self.random_state = random_state
        self.trees = []
        self.feature_indices = []

        if random_state is not None:
            np.random.seed(random_state)

    def bootstrap_sample(self, X, y):
        """Create bootstrap sample"""
        n_samples = len(X)
        indices = np.random.choice(n_samples, n_samples, replace=True)
        return X[indices], y[indices]

    def get_random_features(self, n_features):
        """Select random subset of features"""
        if self.max_features == 'sqrt':
            max_features = int(np.sqrt(n_features))
        elif self.max_features == 'log2':
            max_features = int(np.log2(n_features))
        elif isinstance(self.max_features, int):
            max_features = min(self.max_features, n_features)
```

```
        elif isinstance(self.max_features, float):
            max_features = int(self.max_features * n_features)
        else:
            max_features = n_features

        indices = np.random.choice(n_features, max_features, replace=False)
        return sorted(indices)

    def fit(self, X, y):
        """Train the random forest"""
        n_features = X.shape[1]

        for i in range(self.n_estimators):
            # Bootstrap sampling
            if self.bootstrap:
                X_sample, y_sample = self.bootstrap_sample(X, y)
            else:
                X_sample, y_sample = X, y

            # Random feature selection
            feature_indices = self.get_random_features(n_features)
            self.feature_indices.append(feature_indices)

            # Train decision tree on selected features
            tree = DecisionTreeFromScratch(
                max_depth=self.max_depth,
                min_samples_split=self.min_samples_split,
                criterion='gini'
            )
            tree.fit(X_sample[:, feature_indices], y_sample)
            self.trees.append(tree)

            if (i + 1) % 10 == 0:
                print(f"Trained {i + 1}/{self.n_estimators} trees")

        return self

    def predict(self, X):
        """Make predictions using majority voting"""
        predictions = []

        for tree, feature_indices in zip(self.trees, self.feature_indices):
            pred = tree.predict(X[:, feature_indices])
            predictions.append(pred)

        predictions = np.array(predictions).T

        # Majority voting
        final_predictions = []
        for sample_predictions in predictions:
            final_predictions.append(np.bincount(sample_predictions).argmax())

        return np.array(final_predictions)

    def predict_proba(self, X):
        """Predict class probabilities"""
        predictions = []
```

```python

        for tree, feature_indices in zip(self.trees, self.feature_indices):
            pred = tree.predict(X[:, feature_indices])
            predictions.append(pred)

        predictions = np.array(predictions).T

        # Calculate probabilities
        probas = []
        for sample_predictions in predictions:
            counts = np.bincount(sample_predictions, minlength=2)
            probas.append(counts / len(sample_predictions))

        return np.array(probas)

class RandomForestAnalysis:
    """
    Comprehensive analysis of Random Forest performance
    """

    def __init__(self):
        self.results = {}

    def analyze_ensemble_size(self, X, y):
        """Analyze the effect of number of trees"""
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.3, random_state=42
        )

        n_trees_range = [1, 5, 10, 25, 50, 100, 200]
        train_scores = []
        test_scores = []

        for n_trees in n_trees_range:
            # Custom implementation
            rf_custom = RandomForestFromScratch(
                n_estimators=n_trees,
                max_depth=10,
                random_state=42
            )
            rf_custom.fit(X_train, y_train)

            train_score = accuracy_score(y_train, rf_custom.predict(X_train))
            test_score = accuracy_score(y_test, rf_custom.predict(X_test))

            train_scores.append(train_score)
            test_scores.append(test_score)

            print(f"n_trees={n_trees}: Train={train_score:.3f}, Test={test_score:.3f}")

        # Plot results
        plt.figure(figsize=(10, 6))
        plt.plot(n_trees_range, train_scores, 'o-', label='Training Accuracy', linewidth=2)
        plt.plot(n_trees_range, test_scores, 's-', label='Testing Accuracy', linewidth=2)
        plt.xlabel('Number of Trees')
```

```python
        plt.ylabel('Accuracy')
        plt.title('Random Forest: Effect of Ensemble Size')
        plt.legend()
        plt.grid(True, alpha=0.3)
        plt.xscale('log')
        plt.show()

        return n_trees_range, train_scores, test_scores

    def compare_with_single_tree(self, X, y):
        """Compare Random Forest with single decision tree"""
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.3, random_state=42
        )

        # Single decision tree
        single_tree = DecisionTreeFromScratch(max_depth=10, random_state=42)
        single_tree.fit(X_train, y_train)
        single_tree_train_acc = accuracy_score(y_train, single_tree.predict(X_train))
        single_tree_test_acc = accuracy_score(y_test, single_tree.predict(X_test))

        # Random Forest
        rf = RandomForestFromScratch(n_estimators=100, max_depth=10, random_state=42)
        rf.fit(X_train, y_train)
        rf_train_acc = accuracy_score(y_train, rf.predict(X_train))
        rf_test_acc = accuracy_score(y_test, rf.predict(X_test))

        # Sklearn Random Forest for comparison
        from sklearn.ensemble import RandomForestClassifier
        rf_sklearn = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42)
        rf_sklearn.fit(X_train, y_train)
        rf_sklearn_train_acc = accuracy_score(y_train, rf_sklearn.predict(X_train))
        rf_sklearn_test_acc = accuracy_score(y_test, rf_sklearn.predict(X_test))

        # Create comparison table
        comparison = pd.DataFrame({
            'Model': ['Single Decision Tree', 'Custom Random Forest', 'Sklearn Random Forest'],
            'Train Accuracy': [single_tree_train_acc, rf_train_acc, rf_sklearn_train_acc],
            'Test Accuracy': [single_tree_test_acc, rf_test_acc, rf_sklearn_test_acc],
            'Overfitting Gap': [
                single_tree_train_acc - single_tree_test_acc,
                rf_train_acc - rf_test_acc,
                rf_sklearn_train_acc - rf_sklearn_test_acc
            ]
        })

        print("\nModel Comparison:")
        print("=" * 60)
        print(comparison.round(3))

```

```
        return comparison

    def demonstrate_feature_importance(self, X, y, feature_names=None):
        """Demonstrate feature importance in Random Forest"""
        if feature_names is None:
            feature_names = [f'Feature_{i}' for i in range(X.shape[1])]

        # Train Random Forest
        rf = RandomForestFromScratch(n_estimators=100, max_depth=10, random_state=42)
        rf.fit(X, y)

        # Calculate feature importance (simplified)
        # In a real implementation, we'd track feature usage during training
        feature_usage = np.zeros(X.shape[1])

        # For demonstration, we'll use sklearn's feature importance
        from sklearn.ensemble import RandomForestClassifier
        rf_sklearn = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42)
        rf_sklearn.fit(X, y)

        # Plot feature importance
        plt.figure(figsize=(10, 6))
        indices = np.argsort(rf_sklearn.feature_importances_)[::-1]

        plt.bar(range(len(indices)), rf_sklearn.feature_importances_[indices])
        plt.xticks(range(len(indices)), [feature_names[i] for i in indices], rotation=45)
        plt.xlabel('Features')
        plt.ylabel('Importance')
        plt.title('Random Forest Feature Importance')
        plt.tight_layout()
        plt.show()

        return rf_sklearn.feature_importances_

# Demonstration
if __name__ == "__main__":
    # Create a complex dataset
    X, y = make_classification(
        n_samples=1000, n_features=20, n_informative=15,
        n_redundant=5, n_clusters_per_class=2, random_state=42
    )

    analyzer = RandomForestAnalysis()

    print("Analyzing ensemble size effect...")
    analyzer.analyze_ensemble_size(X, y)

    print("\nComparing with single decision tree...")
    comparison = analyzer.compare_with_single_tree(X, y)

    print("\nDemonstrating feature importance...")
    feature_names = [f'F{i}' for i in range(20)]
    importances = analyzer.demonstrate_feature_importance(X, y, feature_names)

```

```python
    # Show top 10 features
    top_features = np.argsort(importances)[::-1][:10]
    print("\nTop 10 Most Important Features:")
    for i, idx in enumerate(top_features):
        print(f"{i+1}. {feature_names[idx]}: {importances[idx]:.3f}")
```

Listing 4.7: Random Forest Classifier

4.6.3 Boosting Methods

Boosting builds models sequentially, each one correcting its predecessor.

Example

[Boosting Algorithms Comparison]

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import AdaBoostClassifier,
    GradientBoostingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import pandas as pd

class AdaBoostFromScratch:
    """
    Simplified AdaBoost implementation from scratch
    """

    def __init__(self, n_estimators=50, learning_rate=1.0):
        self.n_estimators = n_estimators
        self.learning_rate = learning_rate
        self.models = []
        self.model_weights = []
        self.errors = []

    def fit(self, X, y):
        """Train the AdaBoost classifier"""
        n_samples = len(X)

        # Initialize sample weights
        sample_weights = np.ones(n_samples) / n_samples

        for i in range(self.n_estimators):
            # Train a weak learner (decision stump)
            tree = DecisionTreeClassifier(max_depth=1)
            tree.fit(X, y, sample_weight=sample_weights)

            # Make predictions
            predictions = tree.predict(X)

            # Calculate weighted error
            incorrect = (predictions != y)
            error = np.sum(sample_weights * incorrect) / np.sum(
    sample_weights)
```

```python

            # Calculate model weight
            model_weight = self.learning_rate * np.log((1 - error) / (error + 1e-10))

            # Update sample weights
            sample_weights *= np.exp(model_weight * incorrect)
            sample_weights /= np.sum(sample_weights)  # Normalize

            # Store model and weight
            self.models.append(tree)
            self.model_weights.append(model_weight)
            self.errors.append(error)

            if error == 0:  # Perfect classification
                break

            if (i + 1) % 10 == 0:
                print(f"Trained {i + 1} weak learners, error: {error:.3f}")

        return self

    def predict(self, X):
        """Make predictions using weighted majority voting"""
        # Initialize predictions
        final_predictions = np.zeros(len(X))

        for model, weight in zip(self.models, self.model_weights):
            predictions = model.predict(X)
            final_predictions += weight * predictions

        return np.sign(final_predictions).astype(int)

class BoostingAnalysis:
    """
    Comprehensive analysis of boosting methods
    """

    def __init__(self):
        self.results = {}

    def compare_boosting_algorithms(self, X, y):
        """Compare different boosting algorithms"""
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.3, random_state=42
        )

        # Convert labels to {-1, 1} for our custom AdaBoost
        y_train_ada = np.where(y_train == 0, -1, 1)
        y_test_ada = np.where(y_test == 0, -1, 1)

        # Define algorithms to compare
        algorithms = {
            'Custom AdaBoost': AdaBoostFromScratch(n_estimators=100),
            'Sklearn AdaBoost': AdaBoostClassifier(n_estimators=100, random_state=42),
            'Gradient Boosting': GradientBoostingClassifier(n_estimators=100, random_state=42)
        }
```

```python

        results = {}

        for name, model in algorithms.items():
            print(f"Training {name}...")

            if name == 'Custom AdaBoost':
                model.fit(X_train, y_train_ada)
                train_pred = model.predict(X_train)
                test_pred = model.predict(X_test)

                # Convert back to {0, 1}
                train_pred = np.where(train_pred == -1, 0, 1)
                test_pred = np.where(test_pred == -1, 0, 1)
            else:
                model.fit(X_train, y_train)
                train_pred = model.predict(X_train)
                test_pred = model.predict(X_test)

            train_acc = accuracy_score(y_train, train_pred)
            test_acc = accuracy_score(y_test, test_pred)

            results[name] = {
                'model': model,
                'train_accuracy': train_acc,
                'test_accuracy': test_acc,
                'overfitting_gap': train_acc - test_acc
            }

            print(f"  Train Accuracy: {train_acc:.3f}")
            print(f"  Test Accuracy: {test_acc:.3f}")

        return results

    def analyze_learning_curves(self, X, y):
        """Analyze learning curves for boosting algorithms"""
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.3, random_state=42
        )

        n_estimators_range = [1, 5, 10, 25, 50, 100, 200]

        # Algorithms to analyze
        algorithms = {
            'AdaBoost': AdaBoostClassifier(random_state=42),
            'Gradient Boosting': GradientBoostingClassifier(random_state
=42)
        }

        learning_curves = {}

        for name, model in algorithms.items():
            train_scores = []
            test_scores = []

            for n_estimators in n_estimators_range:
                # Set number of estimators
                if hasattr(model, 'n_estimators'):
                    model.set_params(n_estimators=n_estimators)

```

```
                model.fit(X_train, y_train)

                train_score = accuracy_score(y_train, model.predict(X_train))
                test_score = accuracy_score(y_test, model.predict(X_test))

                train_scores.append(train_score)
                test_scores.append(test_score)

                print(f"{name}, n_estimators={n_estimators}: "
                      f"Train={train_score:.3f}, Test={test_score:.3f}")

            learning_curves[name] = {
                'train_scores': train_scores,
                'test_scores': test_scores
            }

        # Plot learning curves
        plt.figure(figsize=(12, 5))

        for i, (name, curves) in enumerate(learning_curves.items()):
            plt.subplot(1, 2, i+1)
            plt.plot(n_estimators_range, curves['train_scores'], 'o-',
                     label='Training Accuracy', linewidth=2)
            plt.plot(n_estimators_range, curves['test_scores'], 's-',
                     label='Testing Accuracy', linewidth=2)
            plt.xlabel('Number of Estimators')
            plt.ylabel('Accuracy')
            plt.title(f'{name} Learning Curve')
            plt.legend()
            plt.grid(True, alpha=0.3)
            plt.xscale('log')

        plt.tight_layout()
        plt.show()

        return learning_curves

    def demonstrate_boosting_stages(self, X, y):
        """Demonstrate how boosting improves over stages"""
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.3, random_state=42
        )

        # Use a simple 2D dataset for visualization
        from sklearn.datasets import make_moons
        X_simple, y_simple = make_moons(n_samples=300, noise=0.3, random_state=42)
        X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
            X_simple, y_simple, test_size=0.3, random_state=42
        )

        # Train AdaBoost with different numbers of estimators
        n_stages = [1, 5, 10, 50]

        fig, axes = plt.subplots(2, 4, figsize=(16, 8))

        for i, n_estimators in enumerate(n_stages):
            # Train model
```

```python
            ada = AdaBoostClassifier(n_estimators=n_estimators, random_state=42)
            ada.fit(X_train_s, y_train_s)

            # Calculate accuracy
            train_acc = accuracy_score(y_train_s, ada.predict(X_train_s))
            test_acc = accuracy_score(y_test_s, ada.predict(X_test_s))

            # Plot decision boundary
            for j, (X_plot, y_plot, title_suffix) in enumerate([
                (X_train_s, y_train_s, 'Training'),
                (X_test_s, y_test_s, 'Testing')
            ]):
                ax = axes[j, i]

                # Create mesh grid
                x_min, x_max = X_plot[:, 0].min() - 0.5, X_plot[:, 0].max() + 0.5
                y_min, y_max = X_plot[:, 1].min() - 0.5, X_plot[:, 1].max() + 0.5
                xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                                     np.linspace(y_min, y_max, 100))

                Z = ada.predict(np.c_[xx.ravel(), yy.ravel()])
                Z = Z.reshape(xx.shape)

                ax.contourf(xx, yy, Z, alpha=0.3, cmap='RdBu')
                ax.scatter(X_plot[y_plot == 0, 0], X_plot[y_plot == 0, 1],
                           c='blue', label='Class 0', alpha=0.6)
                ax.scatter(X_plot[y_plot == 1, 0], X_plot[y_plot == 1, 1],
                           c='red', label='Class 1', alpha=0.6)

                acc = train_acc if j == 0 else test_acc
                ax.set_title(f'{title_suffix} (n={n_estimators})\nAcc: {acc:.3f}')
                ax.set_xlabel('Feature 1')
                ax.set_ylabel('Feature 2')

        plt.tight_layout()
        plt.show()

# Demonstration
if __name__ == "__main__":
    # Create a complex dataset
    X, y = make_classification(
        n_samples=1000, n_features=10, n_informative=8,
        n_redundant=2, n_clusters_per_class=2, random_state=42
    )

    analyzer = BoostingAnalysis()

    print("Comparing boosting algorithms...")
    results = analyzer.compare_boosting_algorithms(X, y)

    print("\nAnalyzing learning curves...")
    learning_curves = analyzer.analyze_learning_curves(X, y)

    print("\nDemonstrating boosting stages...")
    analyzer.demonstrate_boosting_stages(X, y)
```

```
    # Create results summary
    results_df = pd.DataFrame({
        'Algorithm': list(results.keys()),
        'Train Accuracy': [r['train_accuracy'] for r in results.values()],
        'Test Accuracy': [r['test_accuracy'] for r in results.values()],
        'Overfitting Gap': [r['overfitting_gap'] for r in results.values()
    ]
    })

    print("\nBoosting Algorithms Summary:")
    print("=" * 60)
    print(results_df.round(3))
```

Listing 4.8: Boosting Methods Implementation

4.7 Handling Imbalanced Classification

Real-world datasets often have imbalanced class distributions, requiring special techniques.

4.7.1 Imbalanced Data Challenges

- Majority class dominates learning
- Standard accuracy becomes misleading
- Need for specialized metrics (precision, recall, F1-score)

4.7.2 Techniques for Imbalanced Data

- **Resampling:** Oversampling minority class or undersampling majority class
- **Synthetic Data:** SMOTE (Synthetic Minority Over-sampling Technique)
- **Algorithmic:** Cost-sensitive learning, class weights

Example

[Imbalanced Classification Techniques]

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
     f1_score, confusion_matrix
from sklearn.utils import resample
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from imblearn.combine import SMOTEENN
import pandas as pd
import seaborn as sns

class ImbalancedClassification:
    """
    Comprehensive analysis of imbalanced classification techniques
```

```python
    """

    def __init__(self):
        self.results = {}

    def create_imbalanced_dataset(self, n_samples=1000, imbalance_ratio
=0.1):
        """Create an imbalanced binary classification dataset"""

        n_minority = int(n_samples * imbalance_ratio)
        n_majority = n_samples - n_minority

        # Generate majority class
        X_majority, y_majority = make_classification(
            n_samples=n_majority, n_features=2, n_informative=2,
            n_redundant=0, n_clusters_per_class=1, random_state=42
        )
        y_majority = np.zeros(n_majority, dtype=int)

        # Generate minority class
        X_minority, y_minority = make_classification(
            n_samples=n_minority, n_features=2, n_informative=2,
            n_redundant=0, n_clusters_per_class=1, random_state=43
        )
        # Shift minority class to make it separable
        X_minority += 2
        y_minority = np.ones(n_minority, dtype=int)

        # Combine
        X = np.vstack([X_majority, X_minority])
        y = np.hstack([y_majority, y_minority])

        # Shuffle
        indices = np.random.permutation(len(X))
        X, y = X[indices], y[indices]

        return X, y

    def apply_resampling_techniques(self, X_train, y_train):
        """Apply different resampling techniques"""

        techniques = {}

        # 1. Original (no resampling)
        techniques['Baseline'] = (X_train, y_train)

        # 2. Random Oversampling
        X_majority = X_train[y_train == 0]
        X_minority = X_train[y_train == 1]
        y_majority = y_train[y_train == 0]
        y_minority = y_train[y_train == 1]

        X_minority_upsampled, y_minority_upsampled = resample(
            X_minority, y_minority,
            replace=True,
            n_samples=len(X_majority),
            random_state=42
        )

        X_oversampled = np.vstack([X_majority, X_minority_upsampled])
```

```
        y_oversampled = np.hstack([y_majority, y_minority_upsampled])
        techniques['Random Oversampling'] = (X_oversampled, y_oversampled)

        # 3. Random Undersampling
        rus = RandomUnderSampler(random_state=42)
        X_undersampled, y_undersampled = rus.fit_resample(X_train, y_train)
        techniques['Random Undersampling'] = (X_undersampled, y_undersampled)

        # 4. SMOTE
        smote = SMOTE(random_state=42)
        X_smote, y_smote = smote.fit_resample(X_train, y_train)
        techniques['SMOTE'] = (X_smote, y_smote)

        # 5. SMOTE + ENN
        smote_enn = SMOTEENN(random_state=42)
        X_smote_enn, y_smote_enn = smote_enn.fit_resample(X_train, y_train)
        techniques['SMOTE+ENN'] = (X_smote_enn, y_smote_enn)

        return techniques

    def evaluate_techniques(self, X, y, imbalance_ratio=0.1):
        """Evaluate different imbalanced data handling techniques"""

        # Create imbalanced dataset
        X_imb, y_imb = self.create_imbalanced_dataset(1000, imbalance_ratio)

        # Split data
        X_train, X_test, y_train, y_test = train_test_split(
            X_imb, y_imb, test_size=0.3, random_state=42, stratify=y_imb
        )

        print(f"Original class distribution: {np.bincount(y_imb)}")
        print(f"Training class distribution: {np.bincount(y_train)}")
        print(f"Testing class distribution: {np.bincount(y_test)}")

        # Apply resampling techniques
        techniques = self.apply_resampling_techniques(X_train, y_train)

        # Evaluate each technique
        evaluation_results = {}

        for technique_name, (X_resampled, y_resampled) in techniques.items():
            print(f"\nEvaluating {technique_name}...")
            print(f"Resampled distribution: {np.bincount(y_resampled)}")

            # Train classifier
            clf = RandomForestClassifier(n_estimators=100, random_state=42)
            clf.fit(X_resampled, y_resampled)

            # Predict
            y_pred = clf.predict(X_test)

            # Calculate metrics
            accuracy = accuracy_score(y_test, y_pred)
```

```
            precision = precision_score(y_test, y_pred)
            recall = recall_score(y_test, y_pred)
            f1 = f1_score(y_test, y_pred)

            evaluation_results[technique_name] = {
                'accuracy': accuracy,
                'precision': precision,
                'recall': recall,
                'f1_score': f1,
                'classifier': clf,
                'predictions': y_pred
            }

            print(f"  Accuracy: {accuracy:.3f}")
            print(f"  Precision: {precision:.3f}")
            print(f"  Recall: {recall:.3f}")
            print(f"  F1-Score: {f1:.3f}")

        return evaluation_results, X_test, y_test

    def visualize_results(self, evaluation_results, X_test, y_test):
        """Visualize the results of different techniques"""

        # Create subplots
        fig, axes = plt.subplots(2, 3, figsize=(18, 12))

        # Plot 1: Metrics comparison
        techniques = list(evaluation_results.keys())
        metrics = ['accuracy', 'precision', 'recall', 'f1_score']
        metric_names = ['Accuracy', 'Precision', 'Recall', 'F1-Score']

        x = np.arange(len(techniques))
        width = 0.2

        for i, (metric, metric_name) in enumerate(zip(metrics,
    metric_names)):
            values = [evaluation_results[tech][metric] for tech in
    techniques]
            axes[0, 0].bar(x + i*width, values, width, label=metric_name,
    alpha=0.8)

        axes[0, 0].set_xlabel('Technique')
        axes[0, 0].set_ylabel('Score')
        axes[0, 0].set_title('Performance Metrics Comparison')
        axes[0, 0].set_xticks(x + width*1.5)
        axes[0, 0].set_xticklabels(techniques, rotation=45, ha='right')
        axes[0, 0].legend()
        axes[0, 0].grid(True, alpha=0.3)

        # Plot confusion matrices
        for i, (technique, results) in enumerate(evaluation_results.items
    ()):
            if i < 5:  # Only plot first 5 techniques
                row = (i + 1) // 3
                col = (i + 1) % 3

                cm = confusion_matrix(y_test, results['predictions'])
                sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes
    [row, col])
                axes[row, col].set_title(f'{technique}\nF1: {results["
```

```
f1_score"]:.3f}')
                axes[row, col].set_xlabel('Predicted')
                axes[row, col].set_ylabel('Actual')

        plt.tight_layout()
        plt.show()

        # Create detailed results table
        results_df = pd.DataFrame(evaluation_results).T
        results_df = results_df[['accuracy', 'precision', 'recall', 'f1_score']]

        print("\nDetailed Results:")
        print("=" * 60)
        print(results_df.round(3))

        return results_df

    def demonstrate_class_weights(self, X, y):
        """Demonstrate the use of class weights"""

        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.3, random_state=42, stratify=y
        )

        # Calculate class weights
        from sklearn.utils.class_weight import compute_class_weight
        classes = np.unique(y_train)
        weights = compute_class_weight('balanced', classes=classes, y=y_train)
        class_weights = dict(zip(classes, weights))

        print(f"Class weights: {class_weights}")

        # Compare with and without class weights
        clf_no_weights = RandomForestClassifier(n_estimators=100, random_state=42)
        clf_with_weights = RandomForestClassifier(
            n_estimators=100, random_state=42, class_weight='balanced'
        )

        clf_no_weights.fit(X_train, y_train)
        clf_with_weights.fit(X_train, y_train)

        y_pred_no_weights = clf_no_weights.predict(X_test)
        y_pred_with_weights = clf_with_weights.predict(X_test)

        # Calculate metrics
        metrics_no_weights = {
            'accuracy': accuracy_score(y_test, y_pred_no_weights),
            'precision': precision_score(y_test, y_pred_no_weights),
            'recall': recall_score(y_test, y_pred_no_weights),
            'f1': f1_score(y_test, y_pred_no_weights)
        }

        metrics_with_weights = {
            'accuracy': accuracy_score(y_test, y_pred_with_weights),
            'precision': precision_score(y_test, y_pred_with_weights),
            'recall': recall_score(y_test, y_pred_with_weights),
            'f1': f1_score(y_test, y_pred_with_weights)
```

```python
        }

        comparison = pd.DataFrame({
            'No Class Weights': metrics_no_weights,
            'With Class Weights': metrics_with_weights
        })

        print("\nClass Weights Comparison:")
        print("=" * 60)
        print(comparison.round(3))

        return comparison

# Demonstration
if __name__ == "__main__":
    imb_class = ImbalancedClassification()

    print("Evaluating Imbalanced Classification Techniques...")
    print("=" * 60)

    # Evaluate with severe imbalance
    evaluation_results, X_test, y_test = imb_class.evaluate_techniques(
        None, None, imbalance_ratio=0.1
    )

    # Visualize results
    results_df = imb_class.visualize_results(evaluation_results, X_test,
    y_test)

    # Demonstrate class weights
    print("\nDemonstrating Class Weights...")
    print("=" * 60)

    # Create a balanced dataset for class weights demonstration
    X_balanced, y_balanced = make_classification(
        n_samples=1000, n_features=10, n_informative=8,
        weights=[0.9, 0.1], random_state=42  # Imbalanced classes
    )

    comparison = imb_class.demonstrate_class_weights(X_balanced,
    y_balanced)

    # Show which technique works best for different metrics
    print("\nBest Technique for Each Metric:")
    print("=" * 60)

    best_techniques = {}
    for metric in ['accuracy', 'precision', 'recall', 'f1_score']:
        best_tech = results_df[metric].idxmax()
        best_value = results_df.loc[best_tech, metric]
        best_techniques[metric] = (best_tech, best_value)
        print(f"{metric.capitalize()}: {best_tech} ({best_value:.3f})")
```

Listing 4.9: Handling Imbalanced Data

4.8 Model Evaluation and Comparison

4.8.1 Classification Metrics

- **Accuracy:** Overall correctness
- **Precision:** True positives / (True positives + False positives)
- **Recall:** True positives / (True positives + False negatives)
- **F1-Score:** Harmonic mean of precision and recall
- **ROC-AUC:** Area under Receiver Operating Characteristic curve

4.8.2 Algorithm Comparison Framework

Example

[Classifier Comparison and Evaluation]

```python
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import make_classification, load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_curve, auc,
    precision_recall_curve, roc_auc_score
)
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
import seaborn as sns

class ClassificationEvaluator:
    """
    Comprehensive classification model evaluation and comparison
    """

    def __init__(self):
        self.models = {}
        self.results = {}

    def load_datasets(self):
        """Load and prepare multiple datasets for evaluation"""

        datasets = {}

        # Dataset 1: Simple binary classification
        X_simple, y_simple = make_classification(
            n_samples=1000, n_features=10, n_informative=8,
            n_redundant=2, n_clusters_per_class=1, random_state=42
        )
        datasets['Binary Classification'] = (X_simple, y_simple)

        # Dataset 2: Multi-class classification (Iris)
        iris = load_iris()
        datasets['Iris (Multi-class)'] = (iris.data, iris.target)

```

```python
        # Dataset 3: More complex binary classification
        X_complex, y_complex = make_classification(
            n_samples=1000, n_features=20, n_informative=15,
            n_redundant=5, n_clusters_per_class=2, random_state=42
        )
        datasets['Complex Binary'] = (X_complex, y_complex)

        return datasets

    def initialize_models(self):
        """Initialize various classification models"""

        self.models = {
            'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
            'Decision Tree': DecisionTreeClassifier(max_depth=5, random_state=42),
            'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
            'SVM (Linear)': SVC(kernel='linear', probability=True, random_state=42),
            'SVM (RBF)': SVC(kernel='rbf', probability=True, random_state=42),
            'K-Nearest Neighbors': KNeighborsClassifier(n_neighbors=5)
        }

        return self.models

    def evaluate_model(self, model, X_train, X_test, y_train, y_test, dataset_name):
        """Comprehensive evaluation of a single model"""

        # Train model
        model.fit(X_train, y_train)

        # Predictions
        y_pred = model.predict(X_test)
        y_pred_proba = model.predict_proba(X_test)

        # Calculate metrics
        accuracy = accuracy_score(y_test, y_pred)
        precision = precision_score(y_test, y_pred, average='weighted')
        recall = recall_score(y_test, y_pred, average='weighted')
        f1 = f1_score(y_test, y_pred, average='weighted')

        # Cross-validation scores
        cv_scores = cross_val_score(model, X_train, y_train, cv=5)
        cv_mean = cv_scores.mean()
        cv_std = cv_scores.std()

        # ROC-AUC (for binary classification)
        if len(np.unique(y_test)) == 2:
            roc_auc = roc_auc_score(y_test, y_pred_proba[:, 1])
        else:
            roc_auc = roc_auc_score(y_test, y_pred_proba, multi_class='ovo')

        results = {
            'accuracy': accuracy,
            'precision': precision,
```

```python
            'recall': recall,
            'f1_score': f1,
            'roc_auc': roc_auc,
            'cv_mean': cv_mean,
            'cv_std': cv_std,
            'model': model,
            'predictions': y_pred,
            'probabilities': y_pred_proba
        }

        return results

    def compare_models(self, dataset_name, X, y):
        """Compare all models on a given dataset"""

        # Split data
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.3, random_state=42, stratify=y
        )

        # Standardize features
        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        print(f"\nEvaluating models on {dataset_name}...")
        print("=" * 60)

        dataset_results = {}

        for model_name, model in self.models.items():
            print(f"Training {model_name}...")

            try:
                results = self.evaluate_model(
                    model, X_train_scaled, X_test_scaled, y_train, y_test, dataset_name
                )
                dataset_results[model_name] = results

                print(f"  Accuracy: {results['accuracy']:.3f}")
                print(f"  F1-Score: {results['f1_score']:.3f}")
                print(f"  ROC-AUC: {results['roc_auc']:.3f}")
                print(f"  CV Score: {results['cv_mean']:.3f} (+/- {results['cv_std']:.3f})")

            except Exception as e:
                print(f"  Error: {e}")
                continue

        self.results[dataset_name] = dataset_results
        return dataset_results

    def create_comparison_table(self):
        """Create a comprehensive comparison table"""

        comparison_data = []

        for dataset_name, dataset_results in self.results.items():
            for model_name, results in dataset_results.items():
```

```
                comparison_data.append({
                    'Dataset': dataset_name,
                    'Model': model_name,
                    'Accuracy': results['accuracy'],
                    'Precision': results['precision'],
                    'Recall': results['recall'],
                    'F1-Score': results['f1_score'],
                    'ROC-AUC': results['roc_auc'],
                    'CV Mean': results['cv_mean'],
                    'CV Std': results['cv_std']
                })

        return pd.DataFrame(comparison_data)

    def plot_performance_comparison(self):
        """Create visualizations comparing model performance"""

        fig, axes = plt.subplots(2, 2, figsize=(15, 12))

        # Plot 1: Accuracy comparison across datasets
        dataset_names = list(self.results.keys())
        metrics = ['accuracy', 'f1_score', 'roc_auc', 'cv_mean']
        metric_names = ['Accuracy', 'F1-Score', 'ROC-AUC', 'CV Score']

        for i, (metric, metric_name) in enumerate(zip(metrics,
    metric_names)):
            ax = axes[i // 2, i % 2]

            for dataset_name in dataset_names:
                model_names = list(self.results[dataset_name].keys())
                metric_values = [self.results[dataset_name][model][metric]
                                for model in model_names]

                x_pos = np.arange(len(model_names))
                width = 0.8 / len(dataset_names)

                offset = (len(dataset_names) - 1) * width / 2
                ax.bar(x_pos - offset + dataset_names.index(dataset_name)
    * width,
                      metric_values, width, label=dataset_name, alpha=0.7)

            ax.set_xlabel('Models')
            ax.set_ylabel(metric_name)
            ax.set_title(f'{metric_name} Comparison')
            ax.set_xticks(x_pos)
            ax.set_xticklabels(model_names, rotation=45, ha='right')
            ax.legend()
            ax.grid(True, alpha=0.3)

        plt.tight_layout()
        plt.show()

    def plot_roc_curves(self, dataset_name):
        """Plot ROC curves for binary classification models"""

        if dataset_name not in self.results:
            print(f"No results found for {dataset_name}")
            return

        # Check if it's binary classification
```

```
        results = self.results[dataset_name]
        first_model = list(results.values())[0]
        if len(first_model['probabilities'].shape) == 1 or first_model['probabilities'].shape[1] != 2:
            print("ROC curves are only available for binary classification")
            return

        plt.figure(figsize=(10, 8))

        for model_name, result in results.items():
            fpr, tpr, _ = roc_curve(
                result.get('y_test', np.zeros(len(result['probabilities']))),
                result['probabilities'][:, 1]
            )
            roc_auc = auc(fpr, tpr)

            plt.plot(fpr, tpr, label=f'{model_name} (AUC = {roc_auc:.3f})', linewidth=2)

        plt.plot([0, 1], [0, 1], 'k--', alpha=0.5)
        plt.xlim([0.0, 1.0])
        plt.ylim([0.0, 1.05])
        plt.xlabel('False Positive Rate')
        plt.ylabel('True Positive Rate')
        plt.title(f'ROC Curves - {dataset_name}')
        plt.legend()
        plt.grid(True, alpha=0.3)
        plt.show()

    def plot_confusion_matrices(self, dataset_name):
        """Plot confusion matrices for all models"""

        if dataset_name not in self.results:
            print(f"No results found for {dataset_name}")
            return

        results = self.results[dataset_name]
        n_models = len(results)

        fig, axes = plt.subplots(2, (n_models + 1) // 2, figsize=(15, 10))
        axes = axes.ravel()

        for i, (model_name, result) in enumerate(results.items()):
            if i < len(axes):
                cm = confusion_matrix(result.get('y_test', np.zeros(len(result['predictions']))),
                                      result['predictions'])

                sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[i])
                axes[i].set_title(f'{model_name}\nAcc: {result["accuracy"]:.3f}')
                axes[i].set_xlabel('Predicted')
                axes[i].set_ylabel('Actual')

        # Hide unused subplots
        for i in range(len(results), len(axes)):
            axes[i].axis('off')
```

```python

        plt.tight_layout()
        plt.show()

    def generate_comprehensive_report(self):
        """Generate a comprehensive evaluation report"""

        print("COMPREHENSIVE CLASSIFICATION MODEL EVALUATION REPORT")
        print("=" * 70)

        comparison_df = self.create_comparison_table()

        # Overall best models
        best_models = {}
        for metric in ['Accuracy', 'F1-Score', 'ROC-AUC']:
            best_idx = comparison_df[metric].idxmax()
            best_model = comparison_df.loc[best_idx, 'Model']
            best_dataset = comparison_df.loc[best_idx, 'Dataset']
            best_value = comparison_df.loc[best_idx, metric]
            best_models[metric] = (best_model, best_dataset, best_value)

        print("\nBEST PERFORMING MODELS:")
        print("-" * 50)
        for metric, (model, dataset, value) in best_models.items():
            print(f"{metric}: {model} on {dataset} ({value:.3f})")

        # Model consistency across datasets
        model_consistency = comparison_df.groupby('Model').agg({
            'Accuracy': ['mean', 'std'],
            'F1-Score': ['mean', 'std']
        }).round(3)

        print("\nMODEL CONSISTENCY ACROSS DATASETS:")
        print("-" * 50)
        print(model_consistency)

        return comparison_df

# Demonstration
if __name__ == "__main__":
    evaluator = ClassificationEvaluator()

    # Load datasets
    datasets = evaluator.load_datasets()

    # Initialize models
    evaluator.initialize_models()

    # Evaluate models on all datasets
    for dataset_name, (X, y) in datasets.items():
        evaluator.compare_models(dataset_name, X, y)

    # Generate comprehensive report
    comparison_df = evaluator.generate_comprehensive_report()

    # Create visualizations
    print("\nCreating Performance Visualizations...")
    evaluator.plot_performance_comparison()

    # Plot ROC curves for binary classification datasets
```

```python
    for dataset_name in datasets.keys():
        if 'Binary' in dataset_name:
            print(f"\nPlotting ROC curves for {dataset_name}...")
            evaluator.plot_roc_curves(dataset_name)

    # Plot confusion matrices
    for dataset_name in datasets.keys():
        print(f"\nPlotting confusion matrices for {dataset_name}...")
        evaluator.plot_confusion_matrices(dataset_name)

    # Show detailed comparison table
    print("\nDETAILED COMPARISON TABLE:")
    print("=" * 70)
    pd.set_option('display.max_columns', None)
    pd.set_option('display.width', 1000)
    print(comparison_df.round(3))

    # Additional analysis: Model training time
    print("\nMODEL TRAINING TIME ANALYSIS:")
    print("-" * 50)

    import time
    X, y = datasets['Binary Classification']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
    =0.3, random_state=42)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)

    training_times = {}
    for model_name, model in evaluator.models.items():
        start_time = time.time()
        model.fit(X_train_scaled, y_train)
        end_time = time.time()
        training_times[model_name] = end_time - start_time
        print(f"{model_name}: {training_times[model_name]:.3f} seconds")
```

Listing 4.10: Classification Algorithm Comparison

4.9 Chapter Summary

In this chapter, we covered comprehensive classification techniques:

- **Logistic Regression:** Probabilistic linear classifier using sigmoid function
- **Decision Trees:** Rule-based models that partition feature space recursively
- **Support Vector Machines:** Maximum-margin classifiers with kernel trick
- **Ensemble Methods:** Combining multiple models for improved performance
- **Imbalanced Classification:** Specialized techniques for skewed datasets

Key Insights:

- Different algorithms make different assumptions about data distributions
- Ensemble methods generally outperform individual classifiers Feature scaling is crucial for distance-based algorithms like SVM

- Proper evaluation metrics are essential for imbalanced datasets

4.10 Exercises

4.10.1 Conceptual Questions

1. Explain why logistic regression is considered a linear classifier despite using a non-linear sigmoid function.
2. Compare and contrast the splitting criteria used in decision trees (Gini impurity vs entropy).
3. What is the kernel trick in SVM and why is it particularly useful for non-linear classification?
4. Describe the fundamental differences between bagging (like Random Forests) and boosting (like AdaBoost) ensemble methods.
5. Why might accuracy be a misleading metric for imbalanced classification problems, and what alternatives would you use?

4.10.2 Programming Exercises

1. **Implement Naive Bayes Classifier:**

```python
class NaiveBayesClassifier:
    """Implement Gaussian Naive Bayes from scratch"""
    def fit(self, X, y):
        # Your implementation
        pass

    def predict(self, X):
        # Your implementation
        pass

```

Listing 4.11: Naive Bayes Implementation

2. **Create a Voting Classifier:**

```python
def voting_classifier(models, X_train, y_train, X_test, voting='hard'):
    """
    Combine multiple classifiers using voting
    voting: 'hard' for majority voting, 'soft' for probability averaging
    """
    # Your implementation
    pass

```

Listing 4.12: Voting Classifier

3. **ROC Curve Implementation:**

```python
def plot_roc_curves(classifiers, X_test, y_test):
    """Plot ROC curves for multiple classifiers"""
    # Your implementation
    pass

```

Listing 4.13: ROC Curve Analysis

4.10.3 Lab Assignment: Comprehensive Classification Project

Objective: Build a complete classification pipeline on the Wine Quality dataset.

```python
# Wine Quality Classification Project
import pandas as pd
from sklearn.datasets import fetch_openml

# Load dataset
wine = fetch_openml(name='wine-quality-red', version=1)
X, y = wine.data, wine.target

# Tasks:
# 1. Perform exploratory data analysis
# 2. Handle any data quality issues
# 3. Implement at least 3 different classification algorithms
# 4. Tune hyperparameters for each algorithm
# 5. Compare performance using appropriate metrics
# 6. Create visualizations of decision boundaries (for 2D subsets)
# 7. Write a comprehensive report of your findings
```

Listing 4.14: Classification Project Template

Deliverables:

- Complete Python implementation with comments
- Performance comparison table
- Visualization of results
- Analysis of algorithm strengths and weaknesses
- Recommendations for production deployment